
FPGA Implementation of High Performance Complementary Twin Pyramdial Module Accelerator for Autonomous Vehicles

A dissertation submitted in partial fulfillment of the requireent for the
degree of

Matster of Technology (M.Tech)

by

Chhandak Roy

244102404

Under the Supervision of

Prof. Shaik Rafi Ahamed



DEPARTMENT OF ELECTRONICS AND ELECTRICAL
ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY, GUWAHATI

June , 2026

Declaration

*This is to declare that the thesis entitled “ **FPGA Implementation of High Performance Accelerator for Complementary Twin Pyramdial Module for Autonomous Vehicles**” submitted to the Indian Institute of Technology, Guwahati for the granting of the Master of Technology degree, is a genuine work completed under Prof. Shaik Rafi Ahamed.*

I declare that this thesis is an original work of mine and that this work has not been submitted for any degree or professional qualification. I also wish to state that to the best of my knowledge and understanding, nothing in this report amounts to plagiarism.

Chhandak Roy
Department of Electronics & Electrical Engineering
IIT Guwahati
June 2026, Guwahati

Certificate

*This is to certify that the work contained in the thesis entitled “**FPGA Implementation of High Performance Accelerator for Complementary Twin Pyramdial Module for Autonomous Vehicles**” is a bonafide work of **Chhandak Roy (Roll No: 244102404)**, which has been carried out in the Department of Electronics and Electrical Engineering, Indian Institute of Technology, Guwahati under my supervision and this work has not been submitted elsewhere for a degree. This thesis has met all of the institute’s standards and is suitable for submission.*

Prof. Shaik Rafi Ahamed
Department of Electronics & Electrical Engineering
IIT Guwahati, Assam-781039, India.
June 2026

Acknowledgement

*I would like to extend my sincere thanks to my supervisor **Prof. Shaik Rafi Ahamed** for his constant unwavering support and guidance. Without his support and guidance, it would not have been possible to carry out the project. I also express my gratitude to all the other faculty members of IIT Guwahati's Department of Electronics and Electrical Engineering, whose support and encouragement at important stages gave shape to my project in the present form.*

Furthermore, I would also like to acknowledge the previous works mentioned in the Bibliography section, which were of tremendous help during the course of the project.

Sincerely
Chhandak Roy

Abstract

In the current era of artificial intelligence, transformer-based architectures have revolutionized language, speech, and image understanding due to their powerful global feature modeling capabilities. However, lightweight real-time vision systems deployed on edge hardware still rely heavily on convolutional neural networks (CNNs) because of their lower computational complexity, deterministic execution, reduced memory requirements, and hardware-friendly structure. CNN accelerators remain significantly more practical for FPGA-based embedded systems where power efficiency, latency, and resource utilization are critical constraints. In this context, the proposed **Complementary Twin Pyramidal (CTP)** module becomes highly valuable, improving contextual feature extraction in lightweight segmentation networks while introducing only **0.03 million additional parameters** and achieving nearly **3% improvement in mIoU accuracy**.

Semantic segmentation is one of the most computationally demanding tasks in computer vision because it requires dense pixel-level classification while preserving both fine spatial detail and large-scale contextual information. Although modern CNN-based segmentation models achieve high accuracy, their computational and memory demands make them difficult to deploy on resource-constrained edge devices. In autonomous vehicles and embedded perception systems, GPUs often suffer from high power consumption, thermal limitations, and non-deterministic latency. In contrast, FPGAs provide deterministic low-latency execution, lower power consumption, customizable parallelism, and efficient on-chip memory reuse, making them highly suitable for safety-critical driverless automobile applications.

This project presents a lightweight FPGA-based hardware accelerator for the CTP semantic segmentation module designed for high-throughput real-time inference. The proposed architecture combines pointwise convolution, depthwise dilated convolution, average pooling, bilinear interpolation, batch normalization, ReLU activation, and multi-scale contextual feature fusion within a complementary pyramidal framework. The accelerator employs a deeply pipelined and highly parallel fixed-point architecture optimized specifically for FPGA deployment.

Several architectural optimizations are incorporated, including row-reuse buffering, BRAM-centric (On-chip) memory management, cache-assisted sliding windows, multiplier reuse, pipeline balancing, and multi-output parallel pointwise convolution. The implemented accelerator achieves high-throughput real-time operation while maintaining low DSP and power utilization and moderate LUT consumption, demonstrating the feasibility of deploying lightweight contextual semantic segmentation networks on FPGA platforms for applications such as autonomous driving, robotics, medical imaging, and embedded vision systems.

Contents

1	Introduction	8
1.1	Segmentation	8
1.1.1	Semantic Segmentation	8
1.1.2	Instance Segmentation	8
1.1.3	Panoptic Segmentation	8
1.2	Basics of Semantic Segmentation	9
1.3	CNN Accelerators in Computer Vision	9
1.4	The Need of Light-weight Models	10
1.5	The Complementary Twin Pyramidal (CTP) Module	12
1.6	Key Components of the CTP Module	13
1.6.1	Pointwise Convolution	13
1.6.2	Depthwise Dilated Convolution	14
1.6.3	Batch Normalization and ReLU	15
1.6.4	Average Pooling	16
1.6.5	Global Average Pooling	16
1.6.6	Bilinear Interpolation	17
1.6.7	Pointwise Addition	17
1.7	Applications of Semantic Segmentation	18
2	Literature Survey	19
2.1	Introduction	19
2.2	Lightweight CNN Architectures	19
2.2.1	MobileNet Family	19
2.3	Lightweight Semantic Segmentation Networks	20
2.3.1	BiSeNet and Fast-SCNN	20
2.4	Contextual Feature Extraction Modules	21
2.4.1	ASPP and Pyramid Pooling	21
2.5	FPGA-Based CNN Accelerators	21
2.5.1	CNN-FPGA Co-Design and Dataflow Optimization	21
2.6	Significance to the Proposed Work	22
3	Problem Formulation and Motivation	23
3.1	The Core Problem	23
3.2	Key Architectural Challenges	23
3.3	Motivation for FPGA-Based CTP Acceleration	24
4	Proposed Methodology	25
4.1	System Architecture and Overview	25
4.1.1	STAGE 1: PointWise Convolution(1/4)	26

4.1.2	STAGE 2: Depthwise Conv(6,12) + Average Pooling(GAP,16X16)	27
4.1.3	STAGE 3: Depthwise Conv(18,36) + Average Pooling(8X8,4X4)	29
4.1.4	STAGE 4: PointWise Convolution (1/2)	30
4.2	Fundamental RTL Module	31
4.2.1	Bilinear Interpolation Module	31
4.2.2	Hybrid MAC Module	31
4.2.3	16 Pixel Adder	31
4.2.4	Latency Tracker	31
4.2.5	True Dual-Port BRAM	32
4.3	Design Methodology	33
4.3.1	Eliminating Hardware Division Through Pooling Optimization	33
4.3.2	Pipeline-First Design	34
4.3.3	Memory-Bandwidth-Aware Architecture	34
4.3.4	Parametric Generalisation	39
4.3.5	Parallel Execution and Inter-Stage Overlap	40
4.3.6	Zero-Stall Row Transitions via Buffer-Cache Management	41
4.3.7	Stage 4 Bottleneck Mitigation via Output-Channel Parallelism	43
5	Implementation Results	45
5.1	Post-synthesis FPGA Simulation Results for various Stages	45
5.1.1	STAGE 1	45
5.1.2	STAGE 2	46
5.1.3	STAGE 3	47
5.1.4	STAGE 4	48
5.1.5	FPS Obtained	48
5.2	Hardware Utilization Report	49
5.3	Power Consumption and Frequency Achieved	50
5.3.1	Power Report	50
5.3.2	Timing Report	50
6	Performance Analysis	51
6.1	Experimental Specifications	51
6.2	FLOP and MAC Derivation	51
6.2.1	Pointwise (1×1) Convolution	52
6.2.2	Depthwise Dilated Convolution (DwDConv)	52
6.2.3	Batch Normalisation (Inference)	52
6.2.4	Average Pooling with Window $P \times P$, Stride S	52
6.2.5	Bilinear Interpolation	52
6.3	Stage-by-Stage Computation Budget	53
6.4	Hardware Performance Metrics	54

7	Comparative Analysis	55
7.1	Discussion	56
7.2	Summary	57
7.3	Future Aspects	57

List of Figures

1	Different types of Segmentation	8
2	Block Diagram of the proposed CTP Module.	12
3	1x1 Pointwise Convolution	13
4	3x3 Depthwise Convolution	14
5	Relu Activation function	15
7	Bilinear Interpolation	17
8	CTP Module (updated)	25
9	5 stage Pipelined Stage 1 : PW CONvolution (1/4)	26
10	Stage 2 block 1: Dw Dilated Conv (r=6) with Global Average Pooling	27
11	Stage 2 block 2: DW Dilated Conv (r=12) + Avg Pooling 16x16 . . .	28
12	Stage 3 : Dw dilated Conv (r=18,36) + Avg Pooling 8x8,4x4	29
13	Stage 4 : Pointwise Convolution (1/4)	30
14	True Dual Port BRAM	32
15	CTP Module with updated average Pooling window	33
16	Pre-fetch and Pre-Loading of Cache with Row resue	41
17	STAGE 2 Block1 (DW R=6 + GAP)	42
18	STAGE 4 Pw Convolution(1/2) : 4-way Parallelised Architecture . .	44
19	STAGE 1 Completion Time	45
20	STAGE 2 Completion Time	46
21	STAGE 3 Completion Time	47
22	STAGE 4 Completion Time	48
23	FPS obtained at 160MHz operating frequency	48
24	Hardware Utilization Report CTP accelerator.	49
25	Power Report	50
26	Timing Summary Report	50

1 Introduction

1.1 Segmentation

In computer vision and image processing, semantic segmentation is a fundamental scene understanding task in which every pixel of an image or video frame is assigned a categorical label corresponding to the object or region class it belongs to—such as road, pedestrian, vehicle, building, or sky. Unlike image classification, which produces a single label for the entire image, or object detection, which localises objects within bounding boxes, semantic segmentation provides a dense, pixel-level understanding of the visual scene. There are a few different types of segmentation:

1.1.1 Semantic Segmentation

It is the task of classifying each pixel in an image or frame into one of the predefined set of classes. Once classified, labels will be assigned to every pixel.

1.1.2 Instance Segmentation

It is the task that detects and delineates each instance of an object in an image while labeling each pixel in the image based on the class it belongs to. It can be achieved by combining object detection and semantic segmentation.

1.1.3 Panoptic Segmentation

It is the task that combines both instance segmentation and semantic segmentation. It assigns class label to every pixel in the image and also distinguishes between different individual object instances belonging to the same class.

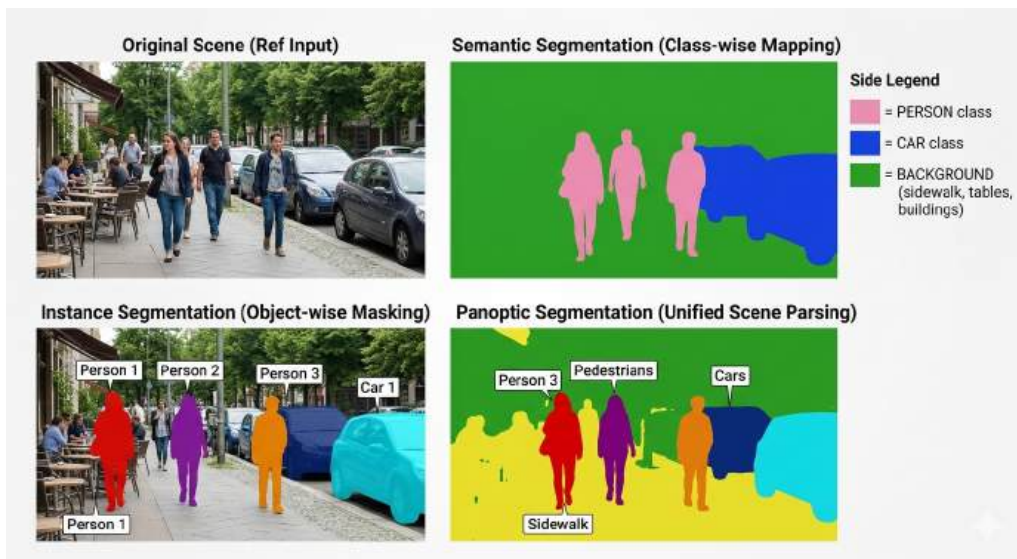


Figure 1: Different types of Segmentation

1.2 Basics of Semantic Segmentation

Semantic segmentation is a fundamental computer vision task that aims to classify every pixel of an image into a predefined semantic category. Unlike image classification, which assigns a single label to an entire image, semantic segmentation produces dense pixel-level predictions. This enables precise scene understanding and spatial awareness.

Object detection draws bounding boxes, image classification assigns a single label. Semantic segmentation does something far more demanding — it produces a dense, pixel-wise prediction map where every single pixel is assigned to a semantic class such as road, pedestrian, sky, or vehicle. The result is a richly structured output that captures not just what objects are present but precisely where they are and what shape they take.

This dense prediction requirement makes semantic segmentation the workhorse of perception systems in autonomous driving (road scene understanding), medical imaging (organ and tumour delineation), aerial image analysis (land cover mapping), and augmented reality (scene-aware overlays). The challenge is that unlike classification, which can extract a global feature vector, segmentation requires the network to maintain fine spatial resolution throughout, yet also capture large-scale contextual clues, making it both memory-intensive and compute-intensive.

1.3 CNN Accelerators in Computer Vision

The rise of deep convolutional neural networks (CNNs) has transformed computer vision. From pioneering architectures like AlexNet (2012) to deeper models such as VGG and ResNet, CNNs have consistently set state-of-the-art benchmarks. However, their immense computational appetite—requiring billions of multiply-accumulate (MAC) operations per single inference—makes real-time deployment on resource-constrained edge devices a serious engineering challenge.

Table 1: Hardware Platform Comparison for Edge Deployment

Metric	CPUs	GPUs	FPGAs
Architecture	Fixed, General Purpose	Fixed, Massively Parallel	Reconfigurable, Custom
Throughput	Low (Sequential execution)	Very High	High (Hardware Pipelined)
Latency	Non-deterministic	High (Batch-dependent)	Low & Deterministic
Power Efficiency	Low	Low (Power-hungry)	Ultra-low (Edge Optimized)

Field-Programmable Gate Arrays (FPGAs) have emerged as a premier platform of choice for hardware acceleration at the edge. Above Structural comparison illustrates architectural trade-offs of FPGA relative to other computing paradigms.

1.4 The Need of Light-weight Models

Full segmentation networks such as DeepLabV3+, PSPNet, or FCN achieve impressive accuracy but require tens of billions of floating-point operations (FLOPs) and hundreds of megabytes of parameters. Consequently, these models are completely infeasible for resource-constrained edge deployment.

Standard convolution operations dominate the computational complexity of CNNs. For a standard convolution:

Computational complexity is proportional to:

$$K^2 \times C_{in} \times C_{out} \times H \times W$$

where:

- K = Kernel size
- C_{in} = Input channels
- C_{out} = Output channels
- H, W = Feature map dimensions

This becomes extremely expensive for high-resolution segmentation tasks. To reduce computational complexity, lightweight CNN architectures replace standard convolutions with:

- Depthwise Convolution
- Pointwise Convolution

Depthwise Convolution:

In depthwise convolution, each input channel is convolved independently using a separate kernel. Complexity becomes:

$$K^2 \times C_{in} \times H \times W$$

Pointwise Convolution:

Pointwise convolution uses 1×1 kernels to combine channel information. Complexity:

$$C_{in} \times C_{out} \times H \times W$$

The field has therefore evolved toward lightweight architectures that trade a small margin of accuracy for dramatic reductions in computational cost. The breakthrough enabling this structural shift was the decomposition of standard convolution into two distinct, computationally cheaper operations **Depthwise** and **Pointwise Convolution**, which **reduces total computation by a staggering factor of approximately 9 times** for typical kernel configurations with minimal accuracy loss.

The proposed **Complementary Twin Pyramidal (CTP)** contextual module directly addresses this challenge by introducing an efficient multi-scale context extraction mechanism specifically designed for lightweight CNN architectures. Instead of relying on computationally expensive backbone expansion or large atrous spatial pyramid modules, CTP combines depthwise dilated convolutions, average pooling, bilinear interpolation, and lightweight feature fusion to capture both local spatial detail and large-scale semantic context with extremely low computational overhead.

A key advantage of the CTP module is that it introduces only approximately **0.03 million additional parameters** while providing nearly **3% improvement in mIoU accuracy**. This makes it highly attractive for resource-constrained edge systems where every DSP block, BRAM resource, memory access, and milliwatt of power is critical. Furthermore, the inherently parallel and depthwise-separable structure of the CTP module maps naturally onto FPGA architectures, making it an ideal solution for real-time semantic segmentation in driverless automobiles and other embedded vision applications.

1.5 The Complementary Twin Pyramidal (CTP) Module

The proposed Complementary Twin Pyramidal Module is an extension of the CTP-Net [12] is a lightweight and hardware-efficient architecture for real-time semantic segmentation on resource-constrained edge platforms. Its fundamental building block, the Complementary Twin Pyramidal (CTP) module, enhances multi-scale feature extraction while substantially reducing computational complexity, memory traffic, and parameter count. The architecture is specifically designed for FPGA deployment, leveraging extensive data reuse, parallelism, and memory-bandwidth-aware optimizations to deliver high throughput with low hardware overhead.

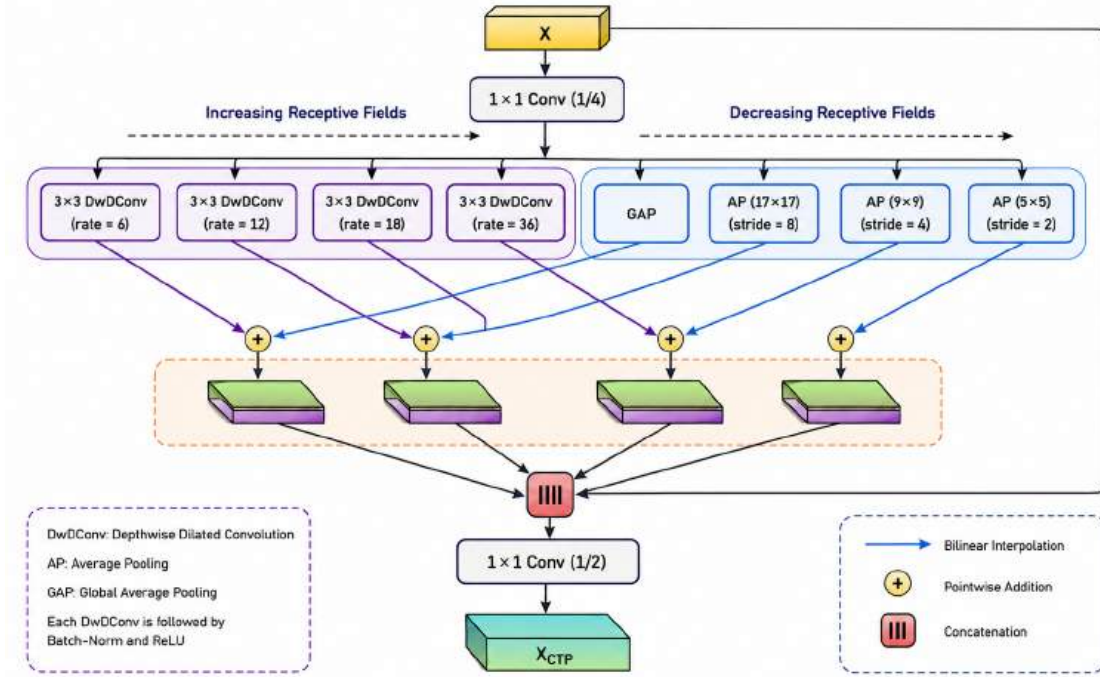


Figure 2: Block Diagram of the proposed CTP Module.

The input feature map X is first processed using a 1×1 pointwise convolution to reduce channel complexity and computational cost. The reduced features are then divided into two complementary pyramidal paths. The first path employs four parallel depthwise dilated convolution branches with dilation rates $r \in \{6, 12, 18, 36\}$ to capture multi-scale contextual information through progressively increasing receptive fields. The second path uses GAP and average pooling branches to extract global semantic information at multiple scales, followed by bilinear interpolation for spatial restoration. Outputs from corresponding branches are fused through pointwise addition, concatenated, and finally projected using a 1×1 convolution to generate the final context-aware feature map X_{CTP} .

1.6 Key Components of the CTP Module

1.6.1 Pointwise Convolution

Pointwise convolution performs channel-wise feature projection using a 1×1 convolution kernel. Unlike spatial convolutions, it operates only along the channel dimension and is primarily responsible for feature mixing and dimensionality reduction. Given an input feature map $X \in \mathbb{R}^{H \times W \times C}$, the output feature map is computed as:

$$Y(i, j, o) = \sum_{c=1}^C X(i, j, c) \cdot W(c, o)$$

where W represents the learnable 1×1 kernel weights. In the proposed CTP module, PW convolution initially reduces the channel depth to one-fourth of the original dimension, significantly lowering the computational complexity of subsequent operations. A final PW convolution is later used for feature fusion and channel projection after concatenation.

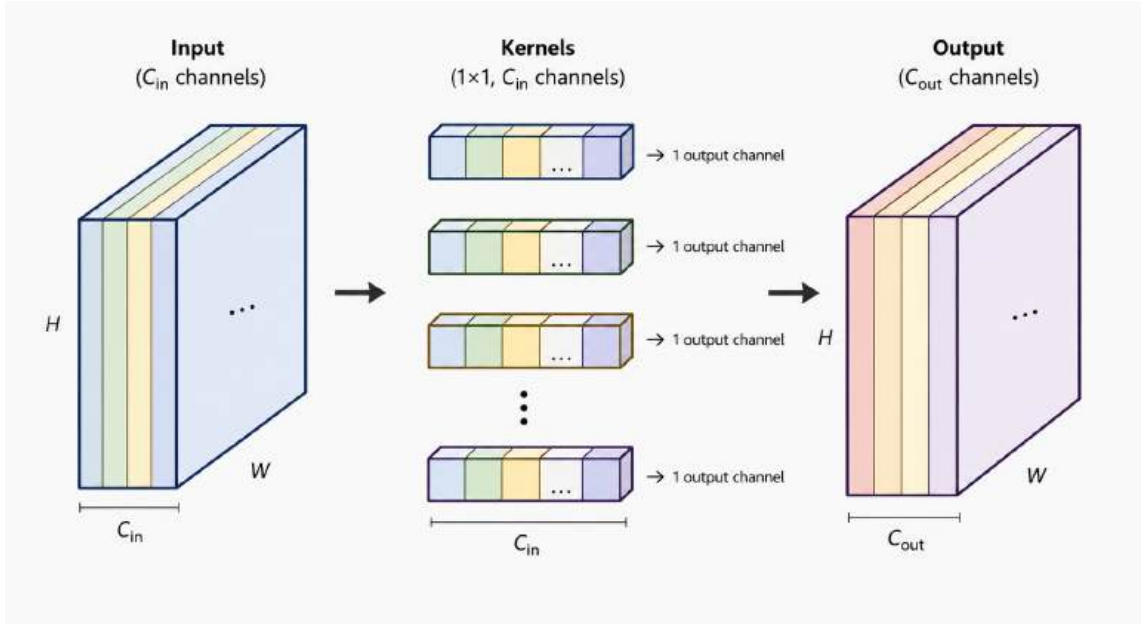


Figure 3: 1x1 Pointwise Convolution

1.6.2 Depthwise Dilated Convolution

Depthwise dilated convolution combines depthwise separable convolution with dilated receptive field expansion. Each input channel is convolved independently using a 3×3 kernel with dilation rate r , increasing the receptive field without increasing parameter count. The operation is expressed as:

$$Y(i, j, c) = \sum_{m=0}^2 \sum_{n=0}^2 X(i + r \cdot m, j + r \cdot n, c) \cdot K(m, n, c)$$

where r denotes the dilation rate. The CTP module employs dilation rates $r \in \{6, 12, 18, 36\}$ to capture multi-scale contextual information ranging from local textures to large semantic structures. Since convolution is applied independently per channel, computational cost remains significantly lower than standard convolution.

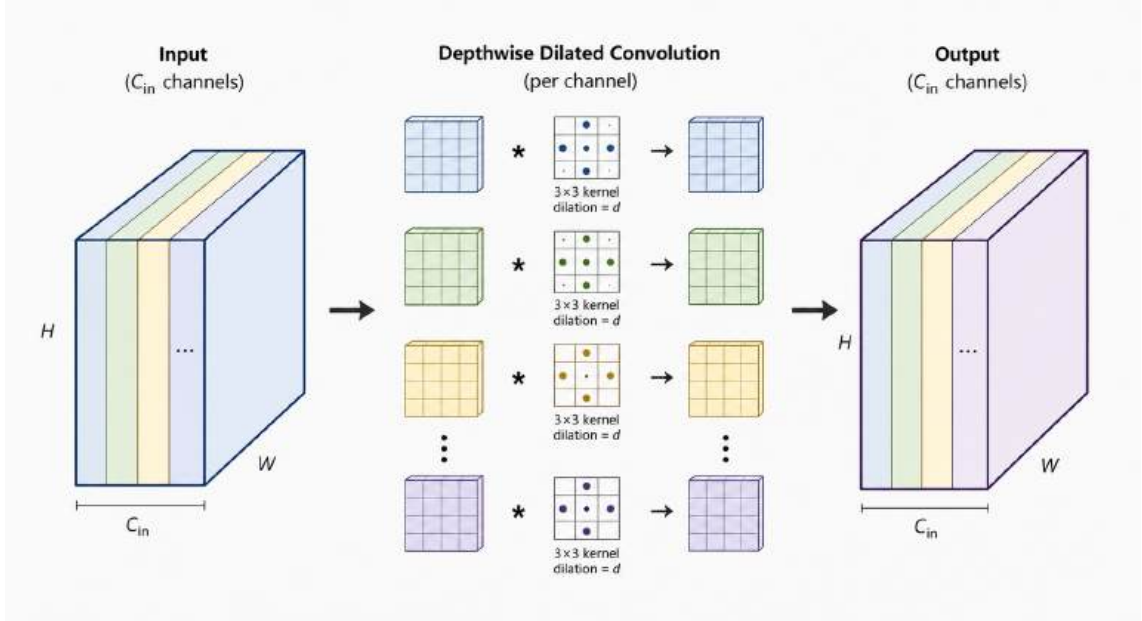


Figure 4: 3x3 Depthwise Convolution

1.6.3 Batch Normalization and ReLU

Batch Normalization stabilizes feature distributions during inference and improves numerical robustness in fixed-point hardware implementations. The normalized output is computed as:

$$Y = \gamma \frac{X - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ and σ^2 are the mean and variance, while γ and β are learnable scale and bias parameters. ReLU activation then introduces non-linearity using:

$$f(x) = \max(0, x)$$

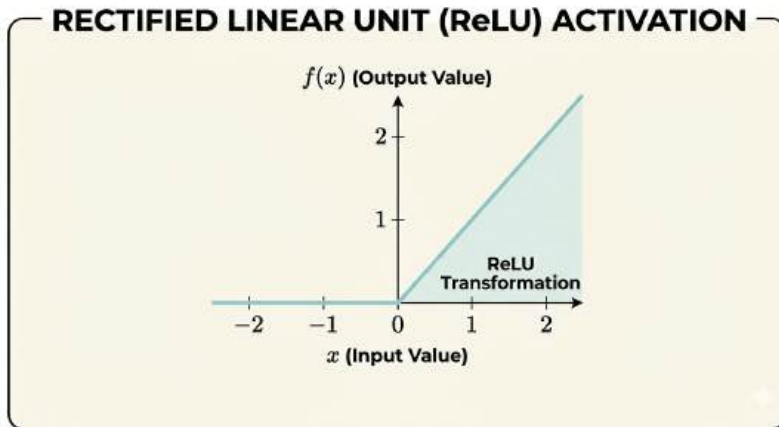


Figure 5: Relu Activation function

1.6.4 Average Pooling

Average pooling reduces spatial resolution while preserving low-frequency semantic information. For a pooling window of size $k \times k$, the pooled output is computed as:

$$Y(i, j) = \frac{1}{k^2} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i + m, j + n)$$

The CTP module uses progressively smaller pooling windows such as 17×17 , 9×9 , and 5×5 with corresponding strides to form a decreasing receptive field pyramid. This branch captures coarse contextual information with minimal computational overhead.

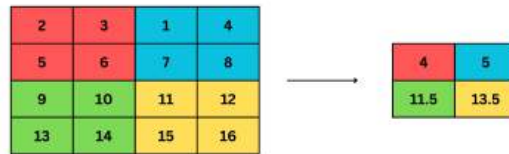
1.6.5 Global Average Pooling

Global Average Pooling compresses an entire feature map into a single scalar value per channel by averaging across all spatial locations:

$$Y(c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X(i, j, c)$$

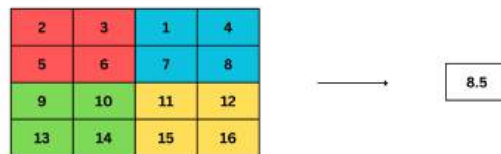
GAP captures global scene-level statistics and semantic context. In the CTP module, the GAP output is broadcast back and fused with dilated convolution features to strengthen global contextual awareness while maintaining negligible parameter overhead.

Average Pooling



Take average of all values in the window

Global Average Pooling



No Concept of Windows. Take Average of all Values in the Matrix.

1.6.6 Bilinear Interpolation

Pooling operations reduce spatial resolution, requiring up-sampling before feature fusion. Bilinear interpolation restores feature-map dimensions using weighted averaging of neighboring pixels:

$$Y(x, y) = \sum_i \sum_j w_{ij} X(i, j)$$

where w_{ij} are interpolation weights determined by relative distances. Bilinear interpolation provides smooth spatial reconstruction with very low hardware complexity, making it highly suitable for FPGA implementation.

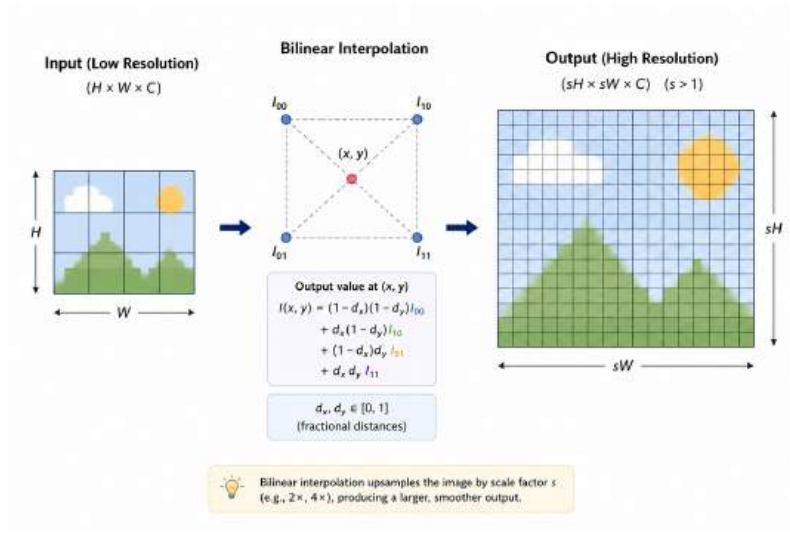


Figure 7: Bilinear Interpolation

1.6.7 Pointwise Addition

Pointwise addition fuses complementary feature maps from the dilated convolution and pooling branches. The fusion operation is defined as:

$$Y(i, j, c) = A(i, j, c) + B(i, j, c)$$

This operation combines fine spatial details from DwDConv branches with global semantic information from pooling branches. The resulting fused features contain both local texture information and large-scale contextual understanding.

1.7 Applications of Semantic Segmentation

Semantic segmentation is pivotal across diverse domains requiring high-fidelity, pixel-level scene parsing and spatial awareness. Key applications include:

- 1. Autonomous Driving** Enables self-driving vehicles to perceive their immediate environment by classifying every pixel into critical structural categories such as drivable roadways, lane markings, pedestrians, cyclists, traffic signs, and surrounding vehicles. This pixel-level granularity is essential for safe trajectory planning and collision avoidance.
- 2. Medical Image Analysis** Assists clinicians by accurately delineating the boundaries of anatomical structures, blood vessels, and abnormalities (such as tumors or lesions) in MRI, CT, and X-ray scans. This automation significantly enhances diagnostic precision and radiation therapy targeting.
- 3. Remote Sensing and Satellite Imagery** Facilitates geographic information system (GIS) analysis by mapping large-scale satellite and aerial photography into structural terrain classes. It is widely used for monitoring urban sprawl, tracking deforestation, land-cover classification, and evaluating environmental degradation.
- 4. Industrial Inspection and Quality Control** Automates manufacturing inspection lines by evaluating products for micro-cracks, surface anomalies, and assembly flaws. Segmenting defective regions at the pixel level allows automated systems to reject faulty components with minimal human intervention.
- 5. Robotic Manipulation and Navigation** Provides autonomous robots and unmanned aerial vehicles (UAVs) with the geometric understanding needed to interact with their surroundings. It assists robotic arms in locating and isolating specific objects for grasping and helps mobile robots map hazard-free navigation paths.
- 6. Intelligent Traffic Systems (ITS)** Enhances urban monitoring through smart surveillance infrastructure. By identifying and tracking lanes, measuring vehicle density, and detecting traffic anomalies or accidents in real time, it provides actionable data for dynamic traffic signal control and city planning.
- 7. Augmented Reality (AR)** Enables seamless blending of virtual assets with the physical world by generating dense pixel maps of the user's environment. This allows virtual objects to interact realistically with real-world surfaces, supporting accurate occlusion where digital assets can pass behind physical structures.

2 Literature Survey

2.1 Introduction

Semantic segmentation is a dense prediction task that requires pixel-level classification while simultaneously preserving fine spatial information and capturing large-scale contextual features. Although modern deep neural networks achieve high segmentation accuracy, their computational complexity and memory requirements make deployment difficult on resource-constrained edge devices. This challenge becomes even more critical in FPGA-based autonomous driving systems, where real-time inference, deterministic latency, low power consumption, and efficient hardware utilization are essential.

To address these challenges, recent research has focused on:

- Lightweight CNN architectures
- Efficient contextual feature extraction
- Real-time semantic segmentation networks
- Hardware-aware CNN acceleration techniques

This section reviews the major architectures and FPGA acceleration strategies that form the theoretical foundation of the proposed FPGA implementation of the Complementary Twin Pyramidal (CTP) module.

2.2 Lightweight CNN Architectures

2.2.1 MobileNet Family

Howard *et al.* introduced **MobileNetV1** [2], which replaced standard convolutions with depthwise separable convolutions:

- Depthwise convolution for spatial filtering
- Pointwise (1×1) convolution for channel mixing

This decomposition reduced computational complexity by nearly $8\times$ to $9\times$ while maintaining competitive ImageNet accuracy. The architecture established depthwise separable convolution as the fundamental building block for lightweight CNNs.

Sandler *et al.* later proposed **MobileNetV2** [3], introducing inverted residual bottlenecks and linear projections to improve feature representation efficiency with minimal computational overhead.

Howard *et al.* further optimized the architecture through **MobileNetV3** [4]., combining neural architecture search (NAS), squeeze-and-excitation attention, and hard-swish activation for improved accuracy-latency tradeoff.

The MobileNet family is highly relevant to this work because the proposed FPGA accelerator extensively employs:

- Depthwise separable convolution
- Lightweight pointwise convolution
- Hardware-efficient feature extraction
- Reduced parameter complexity

These principles directly influence the architecture of the CTP module.

2.3 Lightweight Semantic Segmentation Networks

2.3.1 BiSeNet and Fast-SCNN

Yu *et al.* proposed the **BiSeNet** architecture [5] **Bilateral Segmentation Network (BiSeNet)**, which separates feature extraction into:

- A spatial path for high-resolution detail preservation
- A context path for semantic understanding

BiSeNet achieved approximately:

$$68.4\% \text{ mIoU at } 65 \text{ FPS}$$

on the Cityscapes dataset, demonstrating that separating spatial and semantic processing significantly improves real-time segmentation efficiency.

Poudel *et al.* introduced **Fast-SCNN** [6], employing a “learning-to-downsample” strategy that minimizes redundant full-resolution computation. The network achieved:

$$68.0\% \text{ mIoU at } 123 \text{ FPS}$$

while maintaining extremely low computational complexity.

Both architectures demonstrate that efficient multi-scale contextual fusion is more important than simply increasing network depth. This design philosophy strongly motivates the complementary dual-branch structure of the proposed CTP module.

These architectures demonstrate that lightweight segmentation performance can be significantly improved through efficient contextual modules rather than computationally expensive backbone scaling.

2.4 Contextual Feature Extraction Modules

2.4.1 ASPP and Pyramid Pooling

Chen *et al.* introduced **Atrous Spatial Pyramid Pooling (ASPP)** [7], which employs multiple parallel dilated convolutions with different dilation rates to capture multi-scale contextual information. ASPP significantly improves segmentation accuracy; however, its use of multiple full convolutions creates high computational and memory overhead.

Zhao *et al.* proposed the **Pyramid Pooling Module (PPM)** in PSPNet [8]. in PSPNet, which performs average pooling at multiple spatial scales followed by up-sampling and concatenation. PPM effectively captures global scene-level context while preserving local information.

The proposed CTP module, originally introduced as the Contextual block of CTPNet [12], is conceptually related to both ASPP and PPM. However, instead of relying on computationally expensive standard convolutions, CTP combines depth-wise dilated convolutions, lightweight pooling branches, bilinear interpolation, and pointwise fusion to achieve efficient multi-scale context aggregation with substantially lower computational and memory overhead, making it highly suitable for FPGA deployment.

This makes the architecture substantially more suitable for FPGA-based lightweight deployment.

2.5 FPGA-Based CNN Accelerators

2.5.1 CNN-FPGA Co-Design and Dataflow Optimization

Zhang *et al.* presented one of the earliest systematic CNN-FPGA co-design frameworks [9]. Their work introduced:

- Roofline performance modeling
- Loop tiling
- Loop unrolling
- Memory-bandwidth-aware optimization

for maximizing FPGA utilization and balancing arithmetic throughput with memory bandwidth constraints.

Sze *et al.* [10] provided a comprehensive survey of deep neural network accelerators, introducing major dataflow strategies including:

- Weight-stationary
- Output-stationary

- Row-stationary

These dataflow techniques established the theoretical basis for efficient on-chip memory reuse and pipeline scheduling in CNN accelerators.

Chen *et al.* [11] further explored accelerator design-space optimization , focusing on:

- On-chip buffer allocation
- Parallelism-throughput tradeoff
- Resource scheduling
- Bandwidth optimization

Their work demonstrated that pointwise convolutions in lightweight CNNs often become memory-bandwidth limited rather than compute-limited. This observation directly motivates the multi-output parallel pointwise convolution engine used in the proposed FPGA implementation.

2.6 Significance to the Proposed Work

The reviewed literature establishes the theoretical and architectural foundations for the proposed FPGA-based CTP accelerator. Lightweight CNN architectures demonstrate the effectiveness of depthwise separable convolution for reducing computational complexity, while modern segmentation networks highlight the importance of efficient multi-scale contextual fusion.

Similarly, ASPP and pyramid pooling techniques motivate the contextual design of the CTP module, whereas prior FPGA accelerator research provides essential insights into:

- BRAM-centric memory organization
- Pipeline balancing
- Parallel MAC scheduling
- Dataflow optimization
- Memory-efficient hardware design

The proposed work integrates these concepts into a unified FPGA accelerator capable of efficiently implementing the CTP contextual module for real-time semantic segmentation in autonomous driving and embedded edge-AI systems.

3 Problem Formulation and Motivation

3.1 The Core Problem

Semantic segmentation is a fundamental perception task in autonomous driving, robotics, and intelligent embedded vision systems because it performs dense pixel-level scene understanding. However, achieving both high segmentation accuracy and real-time inference remains a major challenge for edge deployment.

State-of-the-art segmentation networks such as DeepLabV3+, PSPNet, and transformer-based architectures achieve strong contextual understanding through very deep backbones and computationally intensive modules. Although these models perform well on high-end GPUs, they are unsuitable for embedded FPGA platforms due to excessive computational complexity, memory bandwidth demand, high power consumption, and non-deterministic latency.

The challenge becomes even more difficult in autonomous driving systems, where strict real-time constraints, thermal limitations, and energy efficiency are critical. Driverless vehicles require continuous low-latency perception under tight power budgets, making large GPU-based solutions impractical for many edge deployments. In contrast, FPGAs provide deterministic execution, customizable parallelism, efficient on-chip memory reuse, and significantly lower power consumption.

To address this problem, this work focuses on the FPGA implementation of the Complementary Twin Pyramid (CTP) contextual module, a lightweight multi-scale context aggregation architecture designed specifically for efficient semantic segmentation. The CTP module improves contextual feature extraction while introducing only approximately 0.03M additional parameters and achieving nearly 3% improvement in mIoU accuracy, making it highly suitable for lightweight CNN accelerators.

However, implementing CTP efficiently on FPGA is non-trivial because the architecture introduces irregular memory access patterns, multiple parallel pipelines, bilinear interpolation, and multi-scale feature fusion, all of which require careful hardware-aware architectural co-design rather than a direct software-to-hardware translation.

3.2 Key Architectural Challenges

- **Irregular Memory Access in Dilated Convolution:** Depthwise dilated convolutions require simultaneous access to spatially distant rows. For dilation rate $r = 36$, rows $(n, n + 36, n + 72)$ must be available together, creating complex BRAM access and buffering challenges.
- **Pipeline Synchronization Across Parallel Blocks:** Different contextual branches operate concurrently with unequal execution latencies. Synchronizing

DwDConv, pooling, interpolation, and fusion stages without introducing stalls is a major hardware challenge.

- **Large Data Width and Register Complexity:** Naive implementations of wide feature-map operations generate large variable-index multiplexer trees, significantly increasing LUT utilization and routing delay.
- **Bilinear Interpolation Scheduling:** Interpolation can only begin after sufficient feature rows are generated by dilated convolution branches. Incorrect scheduling leads to pipeline bubbles and reduced throughput.
- **Stage-4 Pointwise Convolution Bottleneck:** The final pointwise convolution operates on concatenated multi-scale features with high channel depth, making it the dominant computational bottleneck without aggressive parallelization.

3.3 Motivation for FPGA-Based CTP Acceleration

The CTP module is particularly well-suited for FPGA acceleration due to its structured dataflow and inherent parallelism:

- **Regular MAC-Centric Computation:** Pointwise and depthwise convolutions map efficiently onto FPGA DSP slices and deeply pipelined MAC architectures.
- **Efficient On-Chip Data Reuse:** The same Stage-1 feature maps are reused across multiple contextual branches. FPGA BRAMs enable low-latency on-chip reuse while minimizing expensive external memory transfers.
- **Parallel Multi-Branch Execution:** The four contextual branches of the CTP module are independent and can execute concurrently using parallel compute pipelines.
- **Hardware-Friendly Depthwise Convolution:** Depthwise separable convolutions significantly reduce FLOPs and naturally map to channel-parallel FPGA architectures.
- **Deterministic Real-Time Inference:** Unlike GPU execution, FPGA pipelines provide predictable latency and timing, which is essential for safety-critical autonomous driving systems.

4 Proposed Methodology

4.1 System Architecture and Overview

The CTP Module has been divided into 4 Stages. Each Stage implements specific blocks of the architecture. And relevant Optimization has been done at every stage level for efficient acceleration.

The CTP Module extracts both Contextual and Spatial level detailing by fusing 2 complementary models of DwD Convolution and Average pooling. A lower Dilation rate has low receptive field but greater Spatial detail which is merged with high receptive field and low spatial detail extracting average pooling. This Complementary fusion helps to improve accuracy immensely.

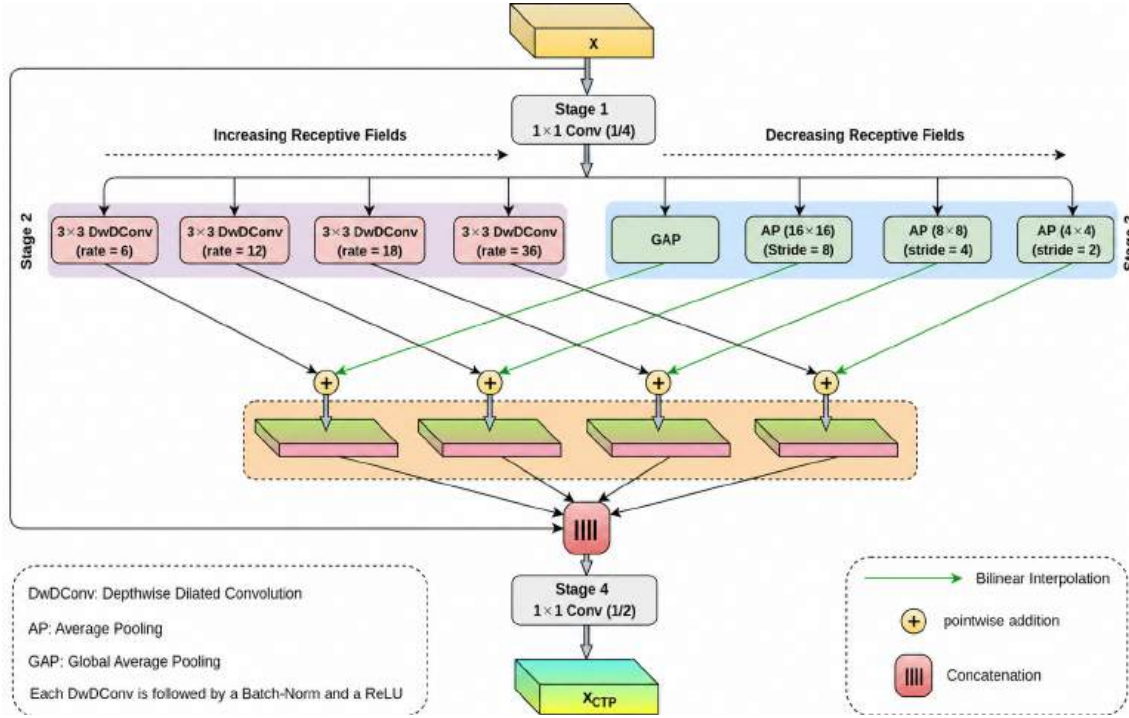


Figure 8: CTP Module (updated)

NOTE: We have replaced the average pooling window sizes from (17×17) , (9×9) , and (5×5) to (16×16) , (8×8) , and (4×4) , respectively. This removes the need for hardware division by enabling shift-based normalization, leading to lower resource utilization and reduced power consumption.

4.1.1 STAGE 1: PointWise Convolution(1/4)

This Stage is one of the Bottleneck of the whole CTP architecture, it takes 31% of total execution time and contributes 10% of overall Computational complexity after Optimization.

We have divided Stage 1 into a fully 5-stage Pipeliend System as follows:

- S1: BRAM Address Updation (Pixel and Kernel Fetching).
- S2: Data arises at BRAM Port (+2 latency)
- S3: Pixel and Kernel Colection from BRAM.
- S4: Multiplying respective Pixels and kernel (128-Parallel Multipliers)
- S5: Accumulating Multiplied Pixels (64-parallel Adders)

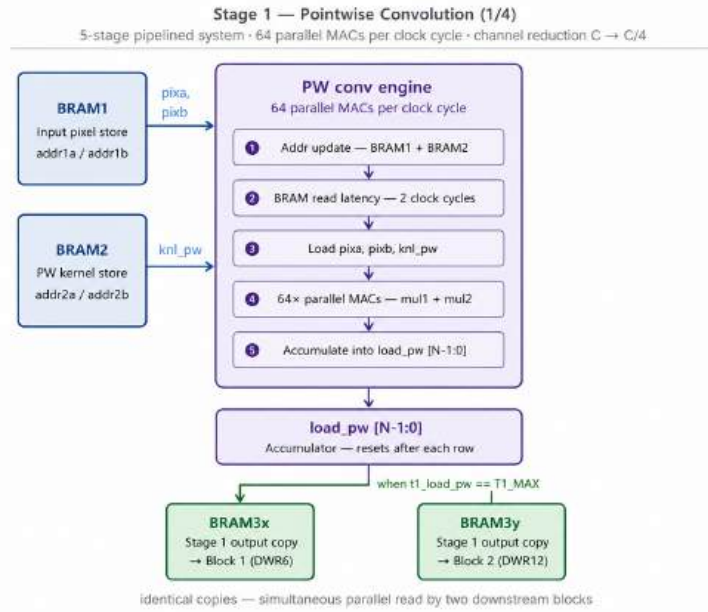


Figure 9: 5 stage Pipleined Stage 1 : PW COnvolution (1/4)

Processing Stage	8 MAC System	2-Row MAC (Ours)	Speed-Up
Single Row Completion	512 cycles	33 cycles	15.5×
Single Channel Completion	65,536 cycles	4,244 cycles	15.5×
Full Image Completion	1,048,576 cycles	67,584 cycles	15.5×
Throughput	2 op pixel/cc	2 op pixel/16cc	16×

Table 2: Speed-up achieved compared to a conventional 8-MAC system for a $128 \times 64 \times 64$ input.

4.1.2 STAGE 2: Depthwise Conv(6,12) + Average Pooling(GAP,16X16)

Stage 2 performs spatial feature extraction through depthwise convolution followed by pooling operations while maximizing data reuse and minimizing intermediate memory transfers.

- **Block 1: DW Dilated Conv (r=6) + Global Average Pooling (GAP)**

Block 1 implements a fused DW dilated convolution and GAP pipeline. Input rows are streamed from BRAM3x into a three-row buffer with cache prefetching, enabling four parallel Hybrid MAC units to execute pipelined 3×3 dilated convolution followed by BN, ReLU, and saturation.

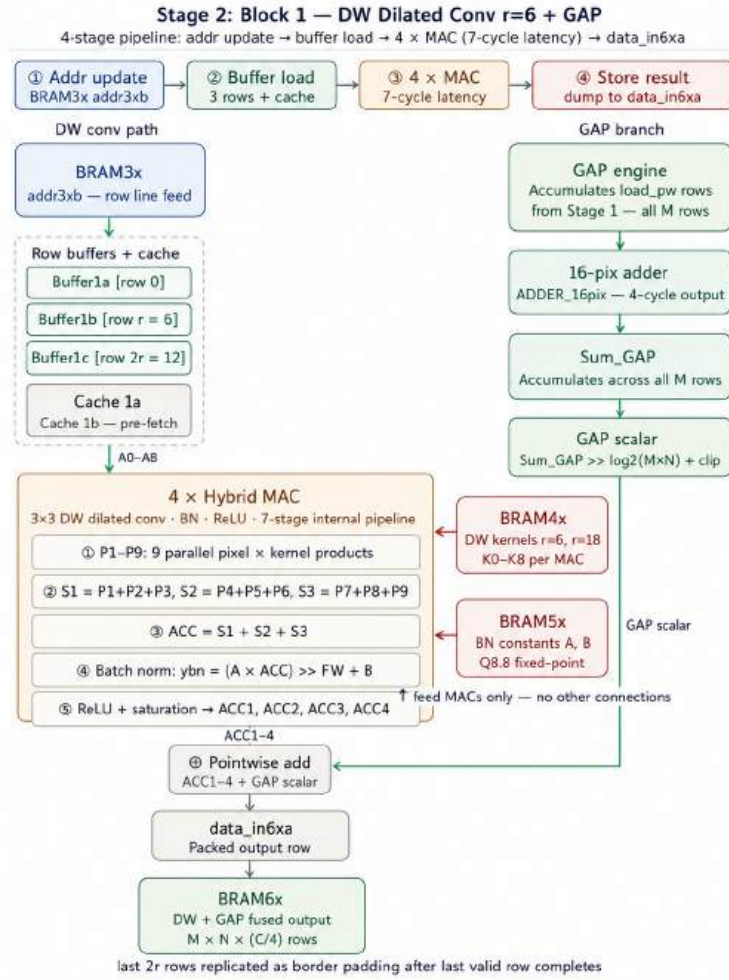


Figure 10: Stage 2 block 1: Dw Dilated Conv (r=6) with Global Average Pooling

The GAP branch operates concurrently by accumulating Stage 1 outputs as soon as each load_pw row becomes available, continuing until all rows of the current channel are processed.

- **Block 2: Dw Dilated Conv (r=12) + Avg Pooling (16×16, stride=8)**

Block 2 follows a similar streaming and reuse-oriented datapath, replacing GAP with average pooling followed by Bilinear upscaling to feature size of Dw Convolution output , after which pointwise addition is done and finally Replication of edge pixels .

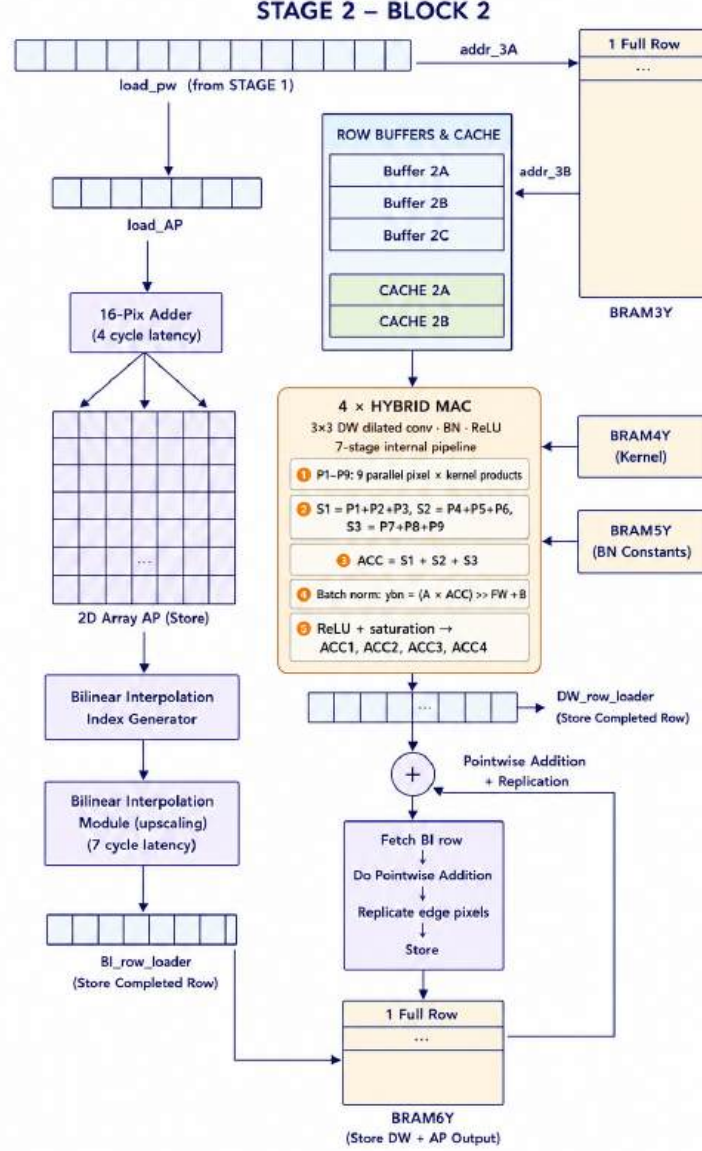


Figure 11: Stage 2 block 2: DW Dilated Conv (r=12) + Avg Pooling 16x16

The row-reuse, Cache and pre-fetch mechanism eliminates row-transition stalls, saving approximately 6 clock cycles per row transition. For a $128 \times 64 \times 64$ input, this corresponds to 672 cycles saved per channel and approximately 10,752 cycles ($\approx 107 \mu s$) per frame.

Note: Stage 2(Block 1 and 2) begins execution in parallel immediately after Stage 1 PW one channel is completed.

4.1.3 STAGE 3: Depthwise Conv(18,36) + Average Pooling(8X8,4X4)

Stage 3 consists of two processing blocks, following an execution pattern analogous to Stage 2 Block 2. Execution of Stage 3 begins only after completion of Stage 2, and the generated outputs are written back into BRAM1. To minimize hardware overhead, Stage 3 reuses the computational resources and storage infrastructure of Stage 2, including modules, BRAMs, registers, and buffering structures. This resource reuse significantly reduces overall hardware utilization while preserving pipeline efficiency.

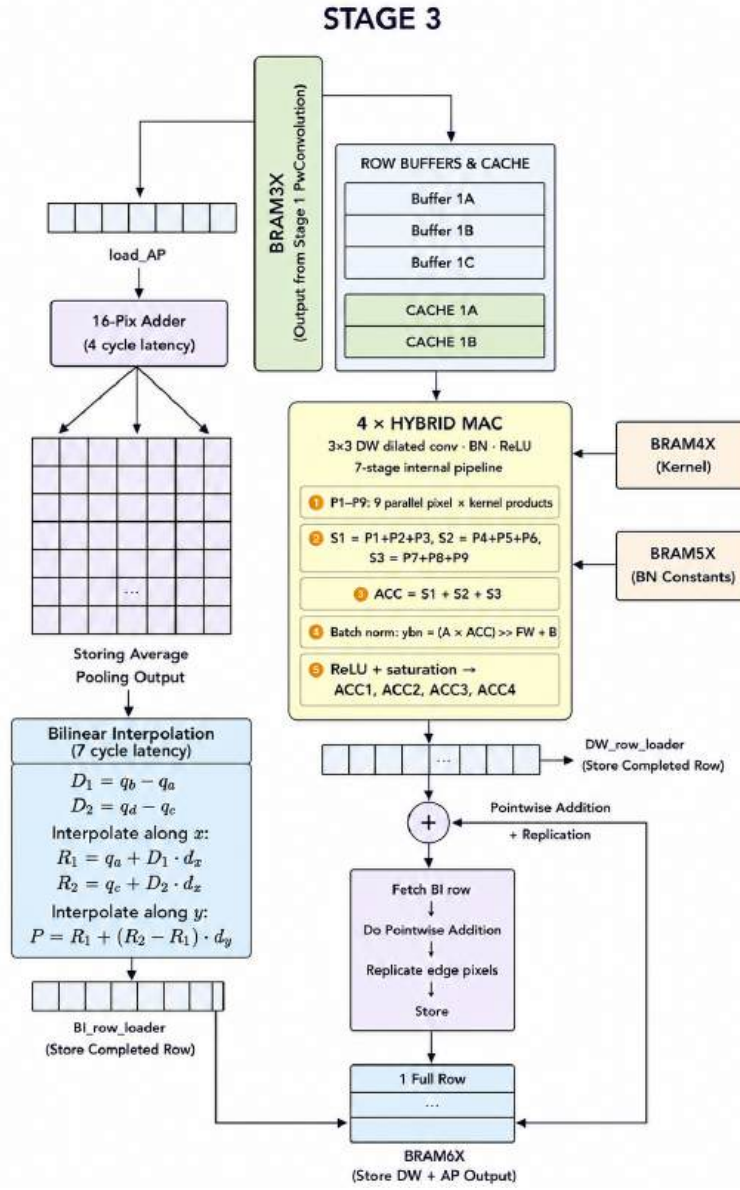


Figure 12: Stage 3 : Dw dilated Conv (r=18,36) + Avg Pooling 8x8,4x4

4.1.4 STAGE 4: PointWise Convolution (1/2)

This stage represents the primary performance **bottleneck of the entire CTP module** architecture, accounting for approximately 80%–90% of the total computational latency. Consequently, extensive optimization efforts were focused on this module through aggressive pipelining and parallelization to achieve higher throughput and lower latency.

By employing a 4-way parallel architecture, utilizing 512 parallel multipliers and 256 adders, and incorporating a 5-stage pipeline within each Pointwise Convolution (PwConv) engine, the computational burden of this stage was significantly reduced. This helped us to Accumulate across 4 channels (2 rows accumulated for each channel parallelly in 1 clock cycle). For an input feature map of size $128 \times 64 \times 64$, these optimizations lowered the bottleneck contribution from **nearly 90% to approximately 61.29%**, resulting in a substantial improvement in overall system performance.

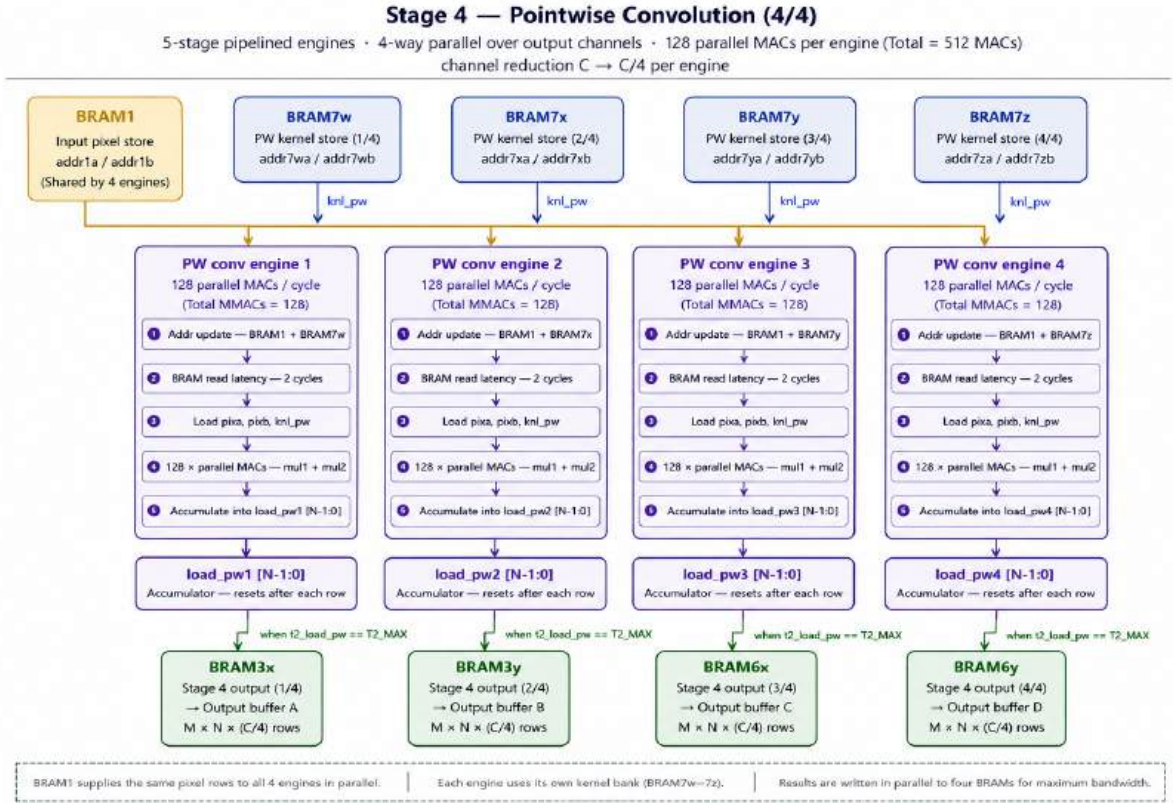


Figure 13: Stage 4 : Pointwise Convolution (1/4)

4.2 Fundamental RTL Module

4.2.1 Bilinear Interpolation Module

Bilinear interpolation is one of the most computationally intensive blocks in the CTP architecture due to its arithmetic complexity and latency requirements. Each interpolation operation uses four neighboring input pixels to generate one output pixel, requiring six multiplication operations and three add/subtract operations in hardware. To achieve the required throughput for feature-map upsampling after Average Pooling in Stage 2 Block 2 and Stage 3, the BI module is instantiated 16 times in parallel. Furthermore, the datapath is heavily pipelined into 8 stages to reduce critical path delay and maintain high operating frequency.

$$\begin{aligned}D_1 &= q_b - q_a \\D_2 &= q_d - q_c \\R_1 &= q_a + \frac{D_1 d_x}{2^F} \\R_2 &= q_c + \frac{D_2 d_x}{2^F} \\P &= R_1 + \frac{(R_2 - R_1) d_y}{2^F}\end{aligned}$$

4.2.2 Hybrid MAC Module

The Hybrid MAC module is implemented as a 7-stage pipelined datapath comprising multiplication, accumulation, batch normalization (BN), and pixel clipping operations. It is the most computationally intensive module in the architecture; however, through deep pipelining and parallel execution, it is capable of producing one output pixel per clock cycle after completing BN, ReLU, and clipping operations.

The Hybrid MAC architecture is primarily utilized in the depthwise dilated convolution stages.

4.2.3 16 Pixel Adder

This block is used in Average pooling and GAP operation. It is 4-Stage pipelined Adder tree which is capable of accumulating 16 pixels.

4.2.4 Latency Tracker

This is one of the most simplest yet crucial module, which helps in synchronization, managing initial pipeline delays, Handshaking between different blocks etc.

Module	LUTs	FFs	DSPs
Bilinear Interpolation	114	116	6
Hybrid MAC	208	162	11
16 pix Adder	251	266	0
Latency tracker	0	Parameter Dependent	0

Table 3: Resource Utilization of basic Modules

4.2.5 True Dual-Port BRAM

True Dual-Port (TDP) Block RAM has been extensively employed in the CTP architecture to address on-chip memory bandwidth limitations and sustain high data throughput across pipeline stages. Unlike single-port memories, TDP BRAM provides simultaneous independent read/write access through two fully functional ports, enabling concurrent data loading and processing without arbitration overhead. This significantly reduces memory stalls and improves overall pipeline efficiency.

To further increase effective bandwidth, data packing (word stacking) has been adopted extensively. For example, for a $128 \times 64 \times 64$ input feature map, one complete image row consisting of 64 pixels with 8-bit precision is packed into a single BRAM word of 512 bits. Since both BRAM ports can operate simultaneously, two complete rows (1024 bits) can be fetched within a single clock cycle, enabling high-parallel MAC execution while minimizing memory access latency.

This combination of true dual-port access and wide-word BRAM organization forms the foundation for achieving sustained throughput and efficient data reuse throughout the CTP datapath.

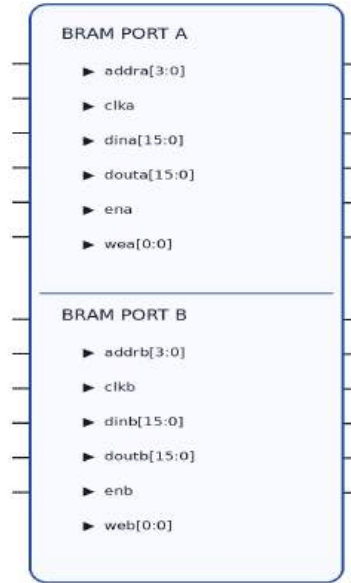


Figure 14: True Dual Port BRAM

4.3 Design Methodology

The FPGA implementation of the CTP module is governed by six interlocking design principles. Each principle is a direct response to a specific challenge identified during architectural analysis: the need for deterministic timing, on-chip bandwidth constraints, hardware scalability, parallel execution across pyramid branches, zero-stall row transitions in dilated convolution, and the elimination of the final-stage throughput bottleneck. Together these principles define a design philosophy that prioritises predictability and resource efficiency over flexibility, which is the correct trade-off for a fixed-function inference accelerator.

4.3.1 Eliminating Hardware Division Through Pooling Optimization

The original CTP architecture employed average-pooling window sizes of 17×17 , 9×9 , and 5×5 . In the proposed implementation, these pooling kernels were redesigned as 16×16 , 8×8 , and 4×4 , respectively. This modification enables the normalization operation to be implemented using simple arithmetic right-shift operations instead of hardware division.

Since division is considerably more resource-intensive and power-consuming than shift-based arithmetic, this optimization significantly reduces logic utilization, lowers power consumption, and simplifies the datapath while preserving the multi-scale contextual feature extraction capability of the original CTP module.

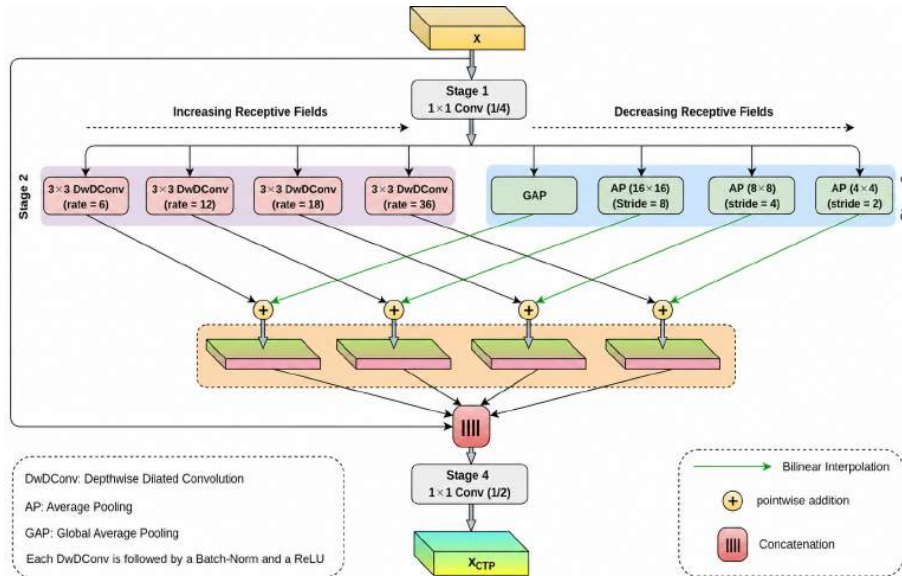


Figure 15: CTP Module with updated average Pooling window

4.3.2 Pipeline-First Design

Every processing block in the CTP implementation is designed as a pipeline from the outset. Rather than computing a result in a monolithic combinational block and tolerating variable completion time, each stage is decomposed into fixed-depth register chains with a known *pipeline latency*—for example, 7 clock cycles for the Hybrid MAC unit and 5 stages for the pointwise convolution engine. Accepting latency as a fixed, known quantity (rather than a variable one) enables the entire system to be scheduled statically.

Handshaking between pipeline stages is managed through a family of lightweight Latency_tracker2 modules. Each tracker is a parameterisable N -stage shift register that propagates a valid_in signal to a valid_out signal exactly N cycles later, with zero logic overhead beyond the shift register itself. Downstream logic—address counters, accumulator enables, BRAM write strobes—is gated exclusively on these registered valid signals. There is no busy-wait polling and no flag-based handshaking that could introduce non-deterministic latency. The result is a fully synchronous, event-driven datapath in which every operation can be traced to a specific clock cycle relative to the triggering event.

Processing Stage / Module	Pipeline Depth (Cycles)
PW Conv Engine	5
PW Conv MAC Unit	2
AP 16-Pixel Adder	4
Hybrid MAC Unit	7
Bilinear Interpolation	7

Table 4: Pipeline latencies within of different module.

4.3.3 Memory-Bandwidth-Aware Architecture

In FPGA-based CNN accelerators, performance is often constrained more by *on-chip memory bandwidth* than by arithmetic throughput. While a DSP can sustain one multiply-accumulate operation per cycle, a BRAM port provides only limited word-level access per cycle. The CTP architecture mitigates this through two mechanisms: *structural bandwidth amplification*, where each BRAM word stores an entire pixel row ($N \times \text{width}$ bits) enabling all pixels to be fetched in a single access, and *temporal BRAM reuse*, where on-chip memories are repurposed across multiple pipeline stages. Consequently, the total on-chip storage remains within the allocated **4 MB** BRAM budget while supporting the four-stage multi-branch pipeline, utilizing only **41.1%** of the total BRAM resources available on the ZCU102 platform.

Design principles. The CTP design pre-empts every foreseeable bandwidth-contention point:

- **Dual output BRAMs (BRAM3x / BRAM3y).** The Stage 1 PW Conv output is written simultaneously to two *identical* BRAMs. Stage 2 Block 1 (DWR6+GAP) reads exclusively from BRAM3x via port B, and Stage 2 Block 2 (DWR12+AP) reads exclusively from BRAM3y via port B, both in the same clock cycle with zero arbitration overhead. Without duplication, the two blocks would share one BRAM, halving effective read bandwidth and stalling one block by half the time.
- **Single-load Kernel and BN-constant BRAMs (BRAM4x / BRAM5x).** Depthwise dilated convolution kernel weights and batch-normalisation scale-bias constants are loaded *once per channel* directly into the Hybrid MAC pipeline registers and never re-fetched during the processing of that channel. This converts a per-pixel weight-read demand into a per-channel amortised cost, reducing BRAM read pressure by a factor equal to the number of pixels processed per channel ($M \times N$).
- **Wide BRAM data words.** Each BRAM address stores one complete pixel row packed into a single $N \times \text{width}$ -bit word. A single BRAM read therefore delivers all N pixel values simultaneously, enabling the 128-way parallel MAC array to be loaded in one cycle rather than N sequential reads.

Address Range	Ch.	Entries	Source	Write Path
$[0, MC - 1]$	C	8192	STAGE 1 PW	Initial Input
$[MC, 5MC/4 - 1]$	$C/4$	2048	S2-B1 (DWR6+GAP)	BRAM6x
$[5MC/4, 6MC/4 - 1]$	$C/4$	2048	S2-B2 (DWR12+AP16x16)	BRAM6y
$[6MC/4, 7MC/4 - 1]$	$C/4$	2048	S3-B3 (DWR18+AP8x8)	BRAM6x
$[7MC/4, 8MC/4 - 1]$	$C/4$	2048	S3-B4 (DWR36+AP4x4)	BRAM6y
Total		16384	STAGE 4 buffer: 1 MB ($\sim 60\%$ BRAM)	

Table 5: BRAM1 Address-space Organization after feature-map reuse for $M = 128$, $N = 64$, $C = 64$. One BRAM address stores one complete row of N pixels.

- **Complete Repurposing of BRAM1 as STAGE 4 Input buffer.** BRAM1 initially stores the C -channel input image ($M \times C$ word entries, one row per word) consumed by Stage 1 PW Conv. Once Stage 1 completes, BRAM1 is *repurposed in-place* to accumulate the concatenated feature tensor that feeds Stage 4, as shown in Table 3. The first $M \times C$ entries already hold the Stage 1 PW output; the remaining $M \times C$ entries are filled in two sequential batches: the Stage 2 outputs and the Stage 3 outputs. The result is a fully populated $2C$ -channel input buffer for Stage 4 PW Conv, obtained with *zero additional*

BRAM allocation beyond what Stage 1 already required.

BRAM 1 is the bottleneck of the on-chip Memory utilization, So it was very necessary to reuse it intensively.

- **Reuse of BRAM6x / BRAM6y across Stages 2 and 3.** BRAM6x and BRAM6y each have depth $M \times (C/4)$. They serve as the output buffers for Stages 2 and 3 in two successive passes:
 1. **Stage 2** (Blocks 1 and 2 execute in parallel): Block 1 (DWR6+GAP) writes its $M \times (C/4)$ output rows into BRAM6x; Block 2 (DWR12+AP 16×16) writes into BRAM6y simultaneously via independent port-A interfaces. Once both blocks complete, the two BRAMs are drained into BRAM1 via their port-B read interfaces, occupying address slots $[M \cdot C \rightarrow 6M \cdot C/4 - 1]$ (two blocks of $M \cdot C/4$ each).
 2. **Stage 3** (Blocks 3 and 4 execute in parallel): The *same* BRAM6x and BRAM6y are immediately reused to buffer Block 3 (DWR18+AP 8×8) and Block 4 (DWR36+AP 4×4) respectively. After Stage 3 completes (BLK3_STATUS), the drain operation repeats, filling BRAM1 slots $[6M \cdot C/4 \rightarrow 2M \cdot C - 1]$ and completing the $2C$ -channel Stage 4 input.

This ping-pong strategy means two BRAM pairs carry the workload of four logical output buffers, halving the BRAM count for intermediate storage.

- **Distributed Stage 4 output across existing BRAMs (BRAM3x, BRAM3y, BRAM6x, BRAM6y).** Stage 4 PW Conv produces C output channels ($= 2C/2$). Rather than allocating new output BRAMs, these C channels are distributed across the four BRAMs that have just been vacated by the upstream stages: BRAM3x and BRAM3y (formerly the Stage 1 output mirrors) each absorb $C/4$ channels, and BRAM6x and BRAM6y (freed after the Stage 3 drain) each absorb another $C/4$ channels. The total capacity of the four BRAMs, $4 \times M \times (C/4) = M \times C$, exactly matches the Stage 4 output volume. No additional storage is therefore allocated for the final feature map X_{CTP} .
- **Quadruplicated Stage 4 kernel BRAMs (BRAM7w/7x/7y/7z) for 4-way output-channel parallelism.** Stage 4 PW Conv operates on $2C$ input channels and produces C output channels, requiring $2C \times C$ kernel values in total ($= 2 \times 16 \times 16 = 512$ for the reference configuration). Processing all C output channels sequentially would make Stage 4 the system bottleneck, consuming **$\sim 85\%$ of total execution time**. To mitigate this, the same $2C \times C$ kernel set is replicated across four dedicated BRAMs (BRAM7w, BRAM7x, BRAM7y, BRAM7z). Each BRAM feeds one of four parallel PW Conv units; every unit processes a distinct group of $C/4$ output channels simultaneously by fetching the relevant kernel slice from its private BRAM. Four units operating concurrently therefore produce four output channels per clock cycle rather than one, reducing Stage 4 latency by $4\times$ at the cost of $4\times$ kernel storage. For the reference configuration this is entirely within the BRAM budget: $4 \times 512 = 2048$ values ≈ 0.06 BRAM tiles, a negligible overhead. The existing Stage 1 kernel BRAM (BRAM2, which holds $C \times (C/4)$ values and is idle after Stage 1) is repurposed to provide bias storage, reclaiming its capacity without additional allocation.

BRAM	Depth (words)	Ports	Contents	Stage Reuse
BRAM1	16384	A/B	Stage1 input + concatenated Stage2/3 output	S1,S4
BRAM2	2048	A	Stage1 PW kernels ($C/4$ filters)	S1 only
BRAM3x	2048	A/B	PW output / Stage4 output ($C/4$ ch)	S1,S2,S3,S4
BRAM3y	2048	A/B	PW output / Stage4 output ($C/4$ ch)	S1,S2,S3,S4
BRAM4x	32	A	DWD kernels (loaded once/channel)	S2,S3
BRAM5x	32	A	BN constants (loaded once/channel)	S2,S3
BRAM6x	2048	A/B	Block1 + Block3 outputs	S2,S3,S4
BRAM6y	2048	A/B	Block2 + Block4 outputs	S2,S3,S4
BRAM7w	8192	A	Stage4 PW kernels (parallel unit 1)	S4
BRAM7x	8192	A	Stage4 PW kernels (parallel unit 2)	S4
BRAM7y	8192	A	Stage4 PW kernels (parallel unit 3)	S4
BRAM7z	8192	A	Stage4 PW kernels (parallel unit 4)	S4

Table 6: Complete On-chip BRAM allocation and reuse strategy for CTP ($M = 128$, $N = 64$, $C = 64$).

Complete on-chip BRAM allocation summary. Table 5 presents the complete On-chip BRAM Organisation of the CTP accelerator for a configuration of $M = 128$, $N = 64$, and $C = 64$. The design extensively reuses BRAM resources across stages to reduce memory footprint while maintaining sufficient bandwidth for parallel execution.

The consequence of this strategy is that *every BRAM tile is occupied for more than one pipeline stage*. A conventional design that allocated separate storage per stage and per branch would require approximately $12 \times M \times (C/4)$ entries of intermediate storage; the reuse scheme reduces this to $2 \times M \times C$ entries for BRAM1 plus four $M \times (C/4)$ buffers — a **3×** reduction in intermediate-buffer BRAM utilisation. Combined with the wide-word fetch strategy, no compute unit ever stalls waiting for data, sustaining the 128-MAC and 512-MAC parallelism in Stages 1 and 4 respectively at full clock-rate throughput.

BRAM	Stored Data	Packing	Memory
BRAM1	S1_PW Conv Input	64 pixels/word	1.00 MB
BRAM2	S1_PW Conv Filters	1 weight/word	1.00 KB
BRAM3x-3y	PW Conv Output	64 pixels/word	128 KB each
BRAM4x-4y	DW Conv Kernels	9 kernels/word	144 B each
BRAM5x-5y	BN Parameters	2 BN constants/word	0.50 KB each
BRAM6x-6y	DW Conv Output/Input	64 pixels/word	128 KB each
BRAM7w-7z	S4_PW Conv Filters	1 kernel/word	8 KB each
Total BRAM Used			1.65 MB
BRAM Utilization (Virtex7: 4.5 MB)			36%

Table 7: On-chip Memory Utilization Report for $128 \times 64 \times 64$ Input image.

4.3.4 Parametric Generalisation

The proposed accelerator is designed as a fully parameterized RTL architecture, allowing seamless scalability across different image resolutions, channel depths, and arithmetic precisions without modifying the core hardware logic. Instead of relying on fixed hard-coded constants, all architectural dimensions, buffer sizes, pipeline widths, and address calculations are automatically derived at synthesis time through Verilog localparam expressions and a synthesizable `logb2()` utility function.

The complete architecture is controlled using the following top-level synthesis parameters:

- **Width** : Defines the bit-width of input feature-map pixels and intermediate data.
- **M** : Represents the input feature-map height (number of rows).
- **N** : Represents the input feature-map width (number of columns).
- **C** : Specifies the number of input feature-map channels.
- **FW** : Defines the fractional precision used for fixed-point BN constants and Bilinear interpolation operation. It decides Accuracy of computation for aforementioned operations.

Using these parameters, all secondary hardware quantities such as accumulator widths, BRAM depths, address bus sizes, loop bounds, tile-cycle counts, saturation limits, and pipeline dimensions are automatically computed during synthesis. This eliminates the use of hard-coded design constants and enables clean architectural portability across multiple configurations.

4.3.5 Parallel Execution and Inter-Stage Overlap

A naive implementation of the four-stage CTP pipeline would execute each stage sequentially: Stage 1 completes all $C/4$ output channels, then Stage 2 begins, and so on. This serialised schedule leaves hardware idle for the majority of the execution window. The CTP design instead exploits two levels of parallelism simultaneously.

Level 1 — Intra-Stage Block Parallelism. Within Stage 2, Block 1 (DW dilated conv $r = 6$ plus GAP) and Block 2 (DW dilated conv $r = 12$ plus AP 16×16) execute *in parallel*. Both blocks read from Stage 1’s output (from BRAM3x and BRAM3y respectively, without contention), process the same input feature map with different spatial operators, and write their outputs to separate BRAMs (BRAM6x and BRAM6y). The same parallelism is replicated in Stage 3, where Block 3 (DWR18 + AP 8×8) and Block 4 (DWR36 + AP 4×4) execute concurrently.

Level 2 — Inter-Stage Pipeline Overlap. Stage 2 does not wait for Stage 1 to finish all $C/4$ output channels. As soon as Stage 1 completes its *first* output channel and writes that channel’s rows to BRAM3x and BRAM3y, Stage 2 Block 1 and Block 2 both begin processing that channel immediately. From this point, Stage 1 and Stage 2 run simultaneously as a producer–consumer pipeline: Stage 1 produces output channels and Stage 2 consumes them, with each block starting on channel k as soon as channel k is available in the shared BRAMs.

Execution timeline and pipeline overlap analysis ($M = 128, N = 64, C = 64$, Input Matrix).

Time	Active Stages	Execution Event
t_0	Stage 1 begins	PW conv starts processing channel 0.
t_1	Stage 1+Stage 2	Channel 0 becomes available (Block 1–2 Starts).
t_2	Stage 1 finishes	Stage 2 (Block 1–2) Starts Parallely.
t_3	Stage 2 finishes	1st Output Loaded in BRAM 1.
t_4	Stage 3(Block 1–2)	Block 1–2 of Stage 3 starts Parallely.
t_5	Stage 3 finishes	2nd batch of output Loaded in BRAM 1.
t_6 onward	Stage 4 begins	Stage 4 begins (4-ch Parallelized execution).

The accelerator employs a streaming producer–consumer schedule where downstream stages start immediately upon availability of intermediate outputs rather than waiting for complete feature-map generation. This maximizes stage concurrency, hides inter-stage latency, and reduces idle compute cycles. Consequently, total execution time approaches a reduction of approximately $C/4$ compared with a serialized execution flow. Parallelization has been employed according to the respective bottlenecks of different stages.

4.3.6 Zero-Stall Row Transitions via Buffer–Cache Management

The depthwise dilated convolution at rate r requires three rows of the input feature map to be simultaneously available: the current row and the rows r and $2r$ positions away. A naive implementation would fetch these rows from BRAM on demand, incurring a 2-cycle BRAM read latency every time the convolution window advances to a new row group—stalling the Hybrid MAC pipeline and destroying throughput.

The CTP design eliminates all stalls through a two-tier *buffer–cache* system:

1. **Active row buffers (Buffer1a, Buffer1b, Buffer1c).** Three packed registers, each $N \times \text{width}$ bits wide, hold the three dilated rows currently being convolved. The Hybrid MAC units read pixel values from fixed bit-slice positions at the LSBs of all three buffers. Every clock cycle, all three buffers are right-shifted by $4 \times \text{width}$ bits, advancing the pixel window by 4 columns and presenting the next four column positions to the four parallel MAC units—at zero combinational cost, since a shift of a packed register is pure wiring in synthesis.
2. **Pre-fetch and Pre-Load cache (Cache1a, Cache1b).** Two additional registers hold the *next* two rows that will be needed after the current row group is exhausted. The cache is loaded from BRAM3x starting exactly $\text{MAX_CYC_DWR6} - 3$ cycles before the current row computation completes—providing a 3-cycle pre-fetch margin that precisely covers the 2-cycle BRAM read latency plus one cycle of register settling. When the current row group finishes, the buffer swap is instantaneous:

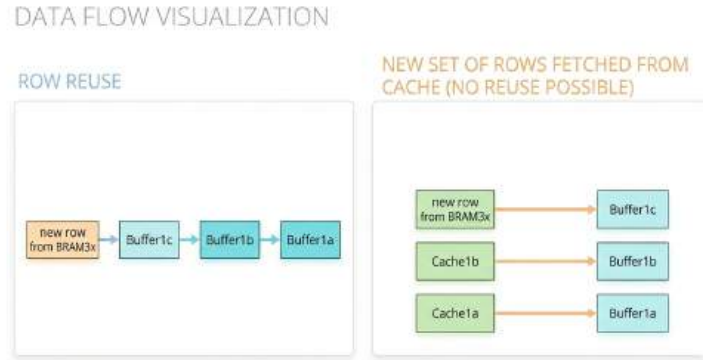


Figure 16: Pre-fetch and Pre-Loading of Cache with Row reuse

The freshly fetched row arrives from BRAM3x in the same clock cycle as the swap (the BRAM address was presented 2 cycles earlier, exploiting the read latency as a *free* pre-fetch period). The Hybrid MAC pipeline therefore never sees an idle cycle between consecutive row groups: the MAC enable signal (Enable_MAC) remains asserted continuously throughout the entire channel.

Stage 2 Block 1 Data Flow and Pipeline. Stage 2 Block 1 realizes a fused DW Dilated Convolution + GAP datapath to improve data reuse and reduce memory transfers. Input rows are streamed from BRAM3x into a three-row buffer with cache prefetching, feeding four parallel Hybrid MAC units executing pipelined 3×3 dilated convolution with BN, ReLU, and saturation. In parallel, the GAP branch accumulates Stage 1 outputs across all M rows using a 16-pixel adder followed by shift-based normalization. The resulting GAP scalar is fused with MAC outputs through pointwise addition and stored into BRAM6x as $M \times N \times (C/4)$ output rows. After the final valid row, the last $2r$ rows are replicated to realize border padding without additional processing overhead.

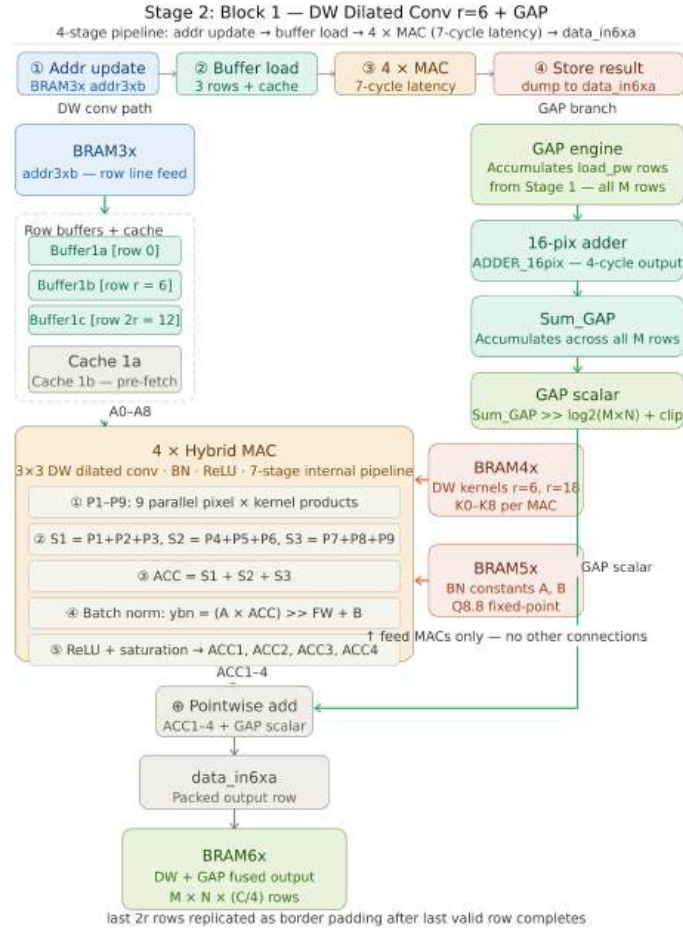


Figure 17: STAGE 2 Block1 (DW R=6 + GAP)

Why this matters. Without the buffer-cache pre-fetch, each row transition would incur a 2-cycle stall for BRAM latency. For the nominal configuration ($M = 64$ rows, $r = 6$, valid rows = 52), there are 52 row transitions per channel and 4 channels per block. Without the cache, $52 \times 4 \times 2 = 416$ stall cycles would be inserted per block—an **overhead of approximately 24% on the 17,000-cycle total frame budget**. The cache system eliminates this overhead entirely at a hardware cost of only two additional row-wide registers per DW Conv block.

4.3.7 Stage 4 Bottleneck Mitigation via Output-Channel Parallelism

Profiling of the execution schedule reveals that Stage 4—the final 1×1 pointwise convolution that reduces the concatenated $2C$ -channel tensor to $C/2$ output channels—accounts for **85–90% of the total frame execution time**. This dominance arises from two compounding factors:

- The input tensor has $2C$ channels (the concatenation of Stage 1 output and the four pyramid block outputs), twice the channel depth of the Stage 1 input. Each output channel requires accumulation over all $2C$ input channels, making Stage 4 $2\times$ more computationally intensive than Stage 1 per output channel.
- Stage 4 produces C output channels (versus $C/4$ in Stage 1), so there are twice as many output channels to compute.

A naively pipelined Stage 4 using a single PW Conv engine (the Stage 1 architecture) would run $4\times$ longer than Stage 1, dominating the execution budget and negating the pipeline overlap gains from Stages 1–3. The design mitigates this through *4-way output-channel parallelism*:

- The Stage 4 PW Conv engine is replicated 4 times, with each instance targeting a disjoint group of output channels. Eight independent kernel BRAMs store the weights for the 4 channel groups; eight independent `load_pw` accumulators accumulate their respective partial sums simultaneously.
- All 4 instances read the *same* input pixel data from BRAM1 (via the same dual-port read). Since the input data is identical for all output channels (only the kernel weights differ), no additional input BRAM bandwidth is required—only $8\times$ the kernel BRAM capacity.
- After 1 input-channel passes, 4 output channels are complete and written in parallel, reducing Stage 4 execution time by $4\times$ relative to the single-instance baseline.

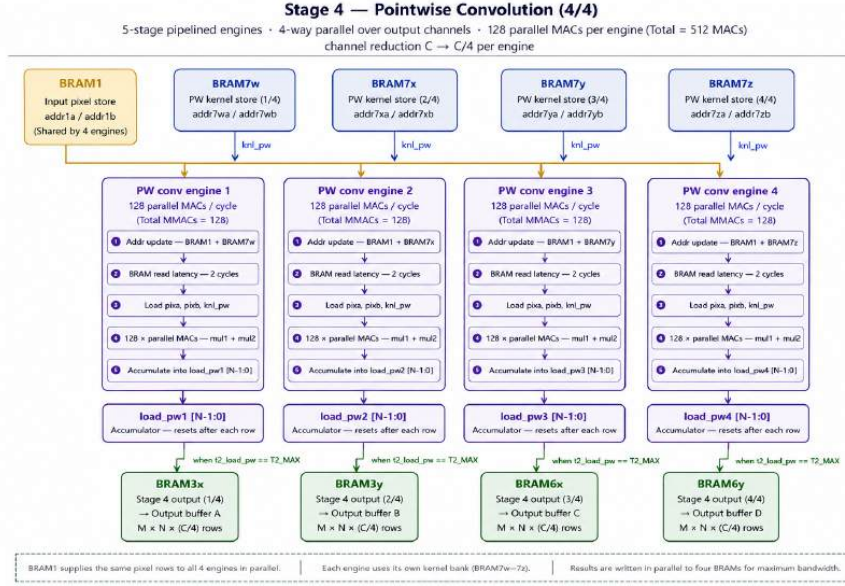


Figure 18: STAGE 4 Pw Convolution(1/2) : 4-way Parallelised Architecture

Stage 4 execution time: with and without 4-way parallelism.

Metric	Single-instance Stage 4	4-way parallel Stage 4
Output channels processed per pass	1	4
Input-channel passes required	C	$C/4$
Cycles relative to Stage 1	$1 \times$ Stage 1	$4 \times$ Stage 1
Share of total frame budget	85–90%	approx. 60–65%
BRAM overhead (kernel)	$1 \times$	$4 \times$
Total Clock cycles to complete	n	$n/4$

These six design principles above are not independent. Pipeline-first design enables parallel execution because all latencies are fixed and known. Memory-bandwidth-awareness enables zero-stall buffering because the cache system's timing can be computed exactly. Parametric generalisation ensures that the 8-way Stage 4 parallelism scales correctly when C is changed. Each principle reinforces the others, and collectively they account for the 794 FPS standalone module throughput and only 12.6% DSP utilisation, 36% BRAM usage reported in the implementation results on Virtex7 vc707 Target Board.

5 Implementation Results

In this Section We present all the Implementation Results that we have obtained in Vvado Xilinx tool from simulation Results to Hardware utilization, Power Consumption, Speed of Operation and Total execution time achieved. All the results are based on the 128x64x64 Image Matrix, which was the typical input to the CTP Module in CTPNet [12] network.

5.1 Post-synthesis FPGA Simulation Results for various Stages

5.1.1 STAGE 1

The first Pointwise Convolution stage completes after **405,585 ns (67,598 clock cycles)**, including the initial reset overhead. This closely matches the theoretical latency estimate of :

$$\frac{M \times C}{4} \left\{ \frac{C}{2} \left(\frac{N + 63}{64} \right) + 1 \right\} = 67,584 \text{ cycles.}$$

resulting in a deviation of only **14 cycles**. The small discrepancy arises from pipeline initialization and reset synchronization overhead at the beginning of execution, thereby validating the accuracy of the proposed cycle-level performance model.

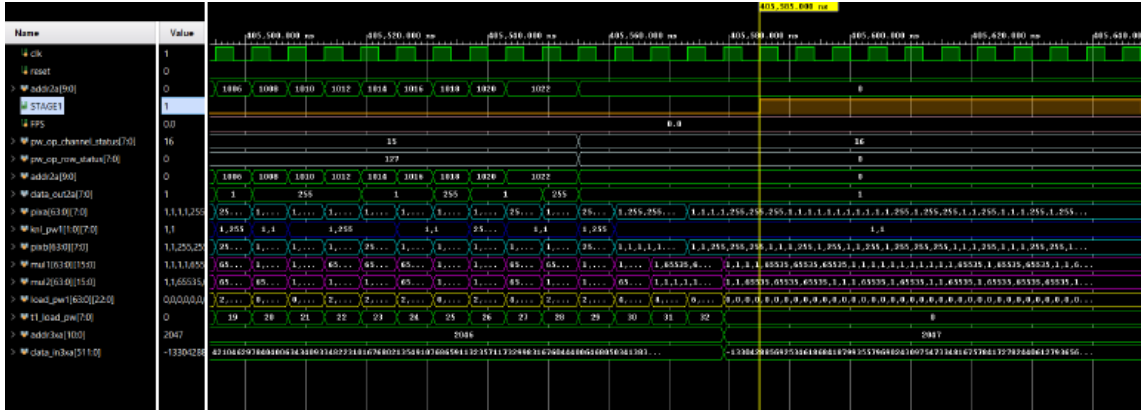


Figure 19: STAGE 1 Completion Time

Therefore, in Stage 1 we almost did around 17 million Floating-point operations in just 67.5k cycles. This is the second biggest bottleneck of the CTP Module, without 128 parallel MAC operation in software it would have take millions of cycles.

5.1.2 STAGE 2

The second stage begins immediately after the completion of the first output channel generated by the Stage 1 pointwise convolution engine, corresponding to approximately 4,224 clock cycles ($\approx 42 \mu s$). Owing to the producer-consumer pipeline organization, Stage 2 executes concurrently with the remaining Stage 1 computations, thereby improving overall throughput. And after Stage 1 completes, Stage 2 finishes at 414,903ns = 69,150 cycles, just 1.8k cycles after stage 1 completes.

Upon completion of Stage 2, the intermediate feature maps stored in BRAM6x and BRAM6y are transferred and concatenated into BRAM1 to form the input tensor for the subsequent processing stage. This BRAM reorganization process requires $128 \times 64 = 8,192$ clock cycles to complete.

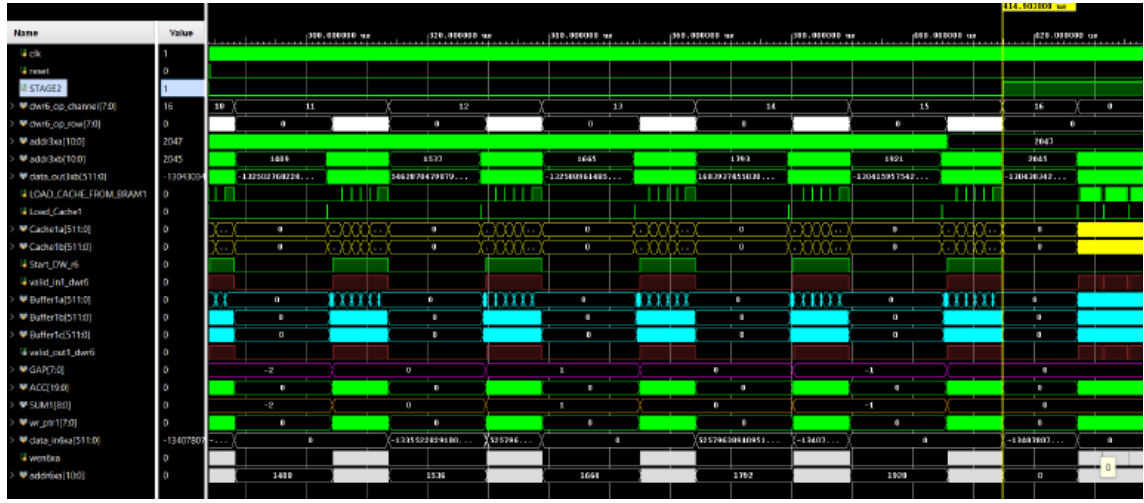


Figure 20: STAGE 2 Completion Time

5.1.3 STAGE 3

After the Stage 2 output results are loaded back to BRAM 1, the Stage 3 begins. Stage 3 feeds on data from BRAM 3y and 3Y. Because of Dw dilated conv with dilation (18,36) it finishes much more quickly than Stage 2, but the pixel replication after a channel is completed takes 2r cycles i.e. 36 cycles for dilation rate=18 and 72 cycles for dilation rate=36. The Stage 3 Completes at 494,907ns, This is a bit larger because of the aforementioned pixel replication and finally sending all the generated pixels from BRAM to BRAM 1, for STAGE 4 to commence.

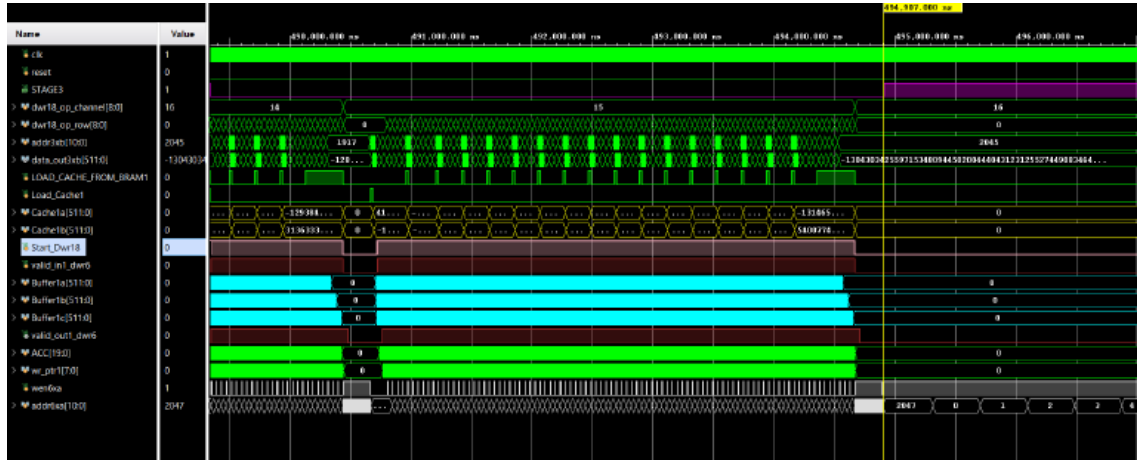


Figure 21: STAGE 3 Completion Time

5.1.4 STAGE 4

Finally, this is the main bottleneck of the entire CTP Module Architecture. It only commences after both Stage 2 and Stage 3 results are stored in BRAM 1 after edge pixels replication.

We can see STAGE 4 completed only in 1,293,651 ns = 215,609 cycles. That means it took approximately 800,000ns to complete which is a massive 61.7% of the total execution Time.

But because of intensive Parallelism and Pipelining, it took comparatively much lesser cycles than other data-flow model that accumulates 4-8 pixels per clock, our model accumulates 512 pixels per clock which makes us 128x to 64x faster, In software, this STAGE 4 would have alone takes 10x more cycle to complete then our entire Execution time of 1,29ms.

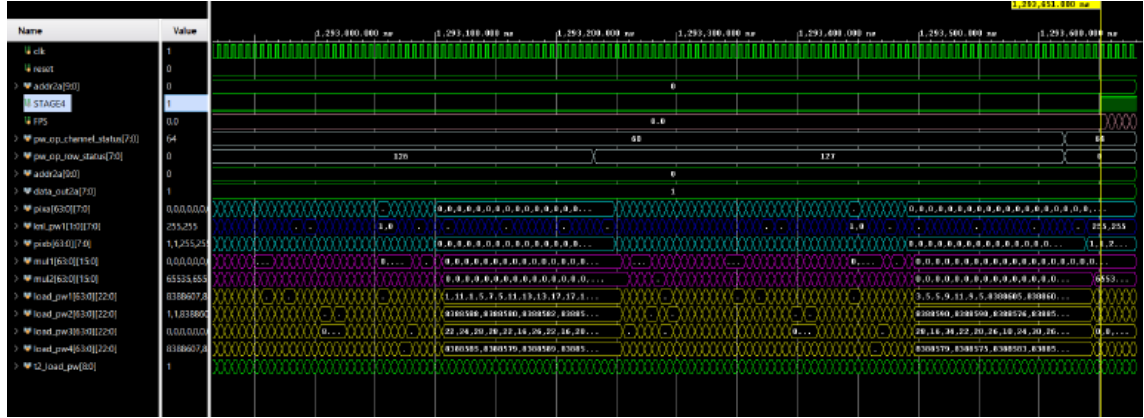


Figure 22: STAGE 4 Completion Time

5.1.5 FPS Obtained

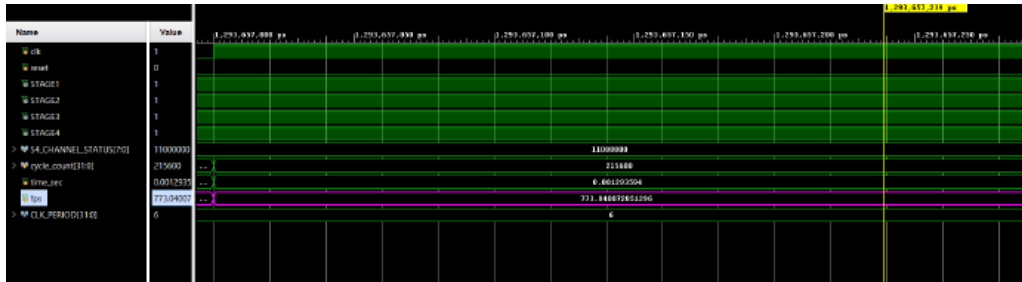


Figure 23: FPS obtained at 160MHz operating frequency

Therefore, we Obtained a **very Low atency of 1.29ms** or around **774 FPS**, after adjusting initial Reset delay. Thus, Our work shows CTP Module which capable of adding 3% accuracy over the baseline can run at ultralow latency.

5.2 Hardware Utilization Report

1. Slice Logic

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	74558	0	0	485760	15.3
LUT as Logic	74558	0	0	134000	55.39
LUT as Memory	0	0	0	46200	0.00
Slice Registers	23507	0	0	607200	3.8
Register as Flip Flop	23507	0	0	607200	3.8
Register as Latch	0	0	0	269200	0.00
F7 Paux	0	0	0	67300	0.00
F8 Paux	0	0	0	33650	0.00

(a) LUTs and FFs Usage

3. Memory

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	370.8	0	0	1030	36
RAMB36/FIFO*	200.5	0	0	510	42.65
RAMB36E1 only	165	0	0	510	0.8
RAMB18	5	0	0	520	0.8
RAMB18E1 only	5	0	0	520	0.8

(b) On-chip Memory Usage

4. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	355	0	0	2800	12.6
DSP48E1 only	355	0	0	2800	12.6

(c) DSP Usage

Figure 24: Hardware Utilization Report CTP accelerator.

Resource	Used	Available	Utilization (%)
LUTs	74,558	485,760	15.3
Flip-Flops (FFs)	23,507	607,200	3.8
BRAM Memory	1.65 MB	4.52 MB	36.0
DSP48E1 Slices	355	2,800	12.6
I/O Pins	14	700	2.0
Total Resource Budget	Within Device Limits		Yes

Table 8: Post-Implementation Resource Utilization Summary of the Proposed CTP Accelerator for a $128 \times 64 \times 64$ Input Tensor

The resource utilization results confirm the hardware efficiency of the proposed CTP accelerator. The design occupies only a modest fraction of the available Virtex-7 resources, with BRAM usage remaining the dominant contributor due to the extensive on-chip buffering employed for high-throughput data reuse. The low LUT, FF, and DSP utilization indicates substantial resource headroom for future architectural extensions while maintaining real-time processing capability.

5.3 Power Consumption and Frequency Achieved

5.3.1 Power Report

Despite achieving a throughput of **774 FPS**, the proposed accelerator consumes only **0.77 W** of on-chip power. This favorable performance-per-watt characteristic is achieved through resource-efficient architectural optimizations that significantly reduce LUT and flip-flop utilization, as discussed in the next section.

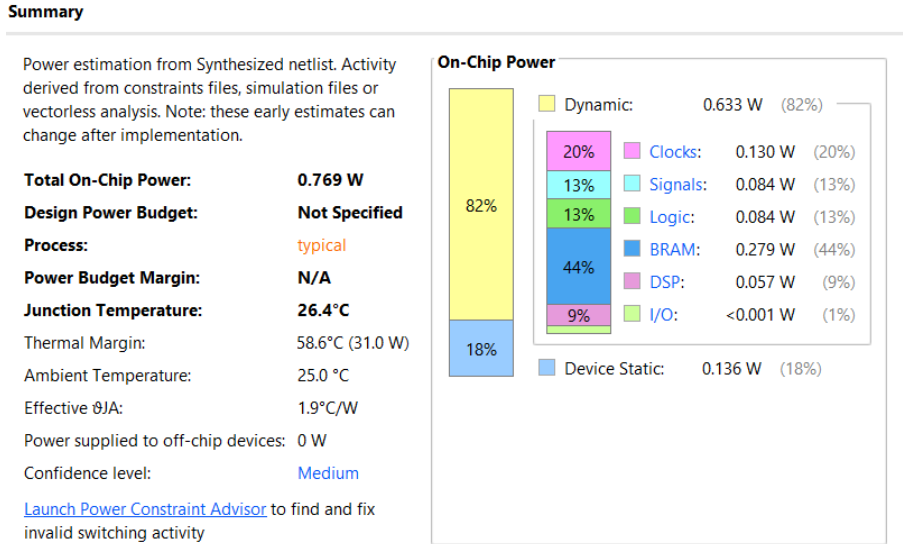


Figure 25: Power Report

5.3.2 Timing Report

The implemented design achieves a timing margin of approximately **7.5 ns** WNS at a target clock period of **12 ns**. This suggests a theoretical maximum operating frequency exceeding **200 MHz**, which has been achieved as a result of efficient Pipelining.

To maintain reliable operation and adequate timing headroom under varying operating conditions, the accelerator is configured to operate at **160 MHz**.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 7.500 ns	Worst Hold Slack (WHS): 0.074 ns	Worst Pulse Width Slack (WPWS): 6.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 89781	Total Number of Endpoints: 89781	Total Number of Endpoints: 23919

All user specified timing constraints are met.

Figure 26: Timing Summary Report

6 Performance Analysis

This section presents a rigorous quantitative characterization of the implemented CTP module accelerator based on the Implementation results. A full derivation of Multiply-Accumulate Operations (MACs), Floating-Point Operations (FLOPs), and Giga-Operations-per-Second (GOP/s) throughput is provided, followed by a hardware resource analysis and a comparative evaluation against published FPGA-based semantic segmentation systems.

6.1 Experimental Specifications

Table 9: Experimental Configuration and Hardware Specifications of the Proposed CTP Accelerator

Parameter	Value	Description
Input Feature Map ($H \times W \times C$)	$128 \times 64 \times 64$	Input spatial resolution and channel depth
Stage 1 PW Convolution Output Channels	$C/4 = 16$	Channel reduction for computational efficiency
Stage 2 & Stage 3 Branch Channels	$C/4 = 16$	Processing width of each contextual branch
Bilinear Interpolation Processing Width	$C/4$	Upsampling of pooled feature maps in each branch
Stage 4 PW Convolution Input Channels	$2C = 128$	Concatenated multi-scale feature representation
Stage 4 PW Convolution Output Channels	$2C/2 = 64$	Final feature projection stage
Target FPGA Platform	Virtex-7 VC707	485,760 LUTs and 2,800 DSP slices
Stage 1 Parallel MAC Units	128	Parallel pointwise convolution engine
Stage 4 Parallel MAC Units	512	Four-channel parallel projection engine
Operating Frequency	160 MHz	Post-implementation operating frequency
Measured Throughput	774 FPS	Maximum sustained frame rate
Measured Latency	1.29 ms	End-to-end processing latency per frame
LUT Utilization	74,558	15.3% of Total LUTs
Power Consumption	0.77Watt	Total On-Chip Power Consumed

6.2 FLOP and MAC Derivation

We adopt the standard definitions used throughout the deep learning accelerator literature: one *MAC* (multiply-accumulate) = one multiplication + one accumulation; one *FLOP* = one floating-point (or fixed-point) arithmetic operation, hence 1 MAC = 2 FLOPs.

6.2.1 Pointwise (1×1) Convolution

For a 1×1 convolution with C_{in} input channels, C_{out} output channels, and spatial dimension $H \times W$:

$$\text{MAC}_{\text{SPW}} = H \times W \times C_{\text{in}} \times C_{\text{out}} \quad (1)$$

$$\text{FLOP}_{\text{SPW}} = 2 \times H \times W \times C_{\text{in}} \times C_{\text{out}} \quad (2)$$

6.2.2 Depthwise Dilated Convolution (DwDConv)

A $K \times K$ depthwise convolution with C channels applies a single $K \times K$ filter per channel independently, regardless of dilation rate r (dilation shifts input positions but does not change the number of operations):

$$\text{MAC}_{\text{SDW}} = H \times W \times C \times K^2 \quad (3)$$

$$\text{FLOP}_{\text{SDW}} = 2 \times H \times W \times C \times K^2 \quad (4)$$

6.2.3 Batch Normalisation (Inference)

During inference, BN reduces to a fused multiply-add $y = \gamma x + \beta$ per element:

$$\text{FLOP}_{\text{SBN}} = 2 \times H \times W \times C \quad (5)$$

6.2.4 Average Pooling with Window $P \times P$, Stride S

Output spatial dimensions: $H_o = \lfloor (H - P)/S \rfloor + 1$, $W_o = \lfloor (W - P)/S \rfloor + 1$. Each output requires P^2 additions plus one right-shift (dividing by P^2 when P is a power of two, hence zero-cost):

$$\text{FLOP}_{\text{SAP}} = H_o \times W_o \times C \times (P^2 - 1) \quad (6)$$

Note: $P \in \{16, 8, 4\}$ (powers of two) allows the division step to be replaced by bit-shifts, eliminating costly division hardware entirely.

6.2.5 Bilinear Interpolation

Total 3 Bilinear interpolation has to be used. Each upsampled output pixel requires 3 multiplications and 6 additions (≈ 9 FLOPs):

$$\text{FLOP}_{\text{SBIL}} = 9 \times H \times W \times C \times 3 \quad (7)$$

6.3 Stage-by-Stage Computation Budget

Applying the formulae above to the target configuration ($H = 128$, $W = 64$, $C = 64$, $C/4 = 16$, $K = 3$):

Table 10: Stage-wise computational cost of the CTP module ($H=128$, $W=64$, $C=64$).

Stage	Component	MACs (M)	FLOPs (M)
Stage 1	PW Conv ($C_{\text{in}}=64 \rightarrow C_{\text{out}}=16$)	8.39	16.78
Stage 2 – Blk 1	DwDConv 3×3 , $d=6$, $C=16$	1.18	2.36
Stage 2 – Blk 2	DwDConv 3×3 , $d=12$, $C=16$	1.18	2.36
Stage 3 – Blk 3	DwDConv 3×3 , $d=18$, $C=16$	1.18	2.36
Stage 3 – Blk 4	DwDConv 3×3 , $d=36$, $C=16$	1.18	2.36
Stages 2–3	BN + ReLU (4 branches, $C=16$)	—	1.05
Stage 2 – Blk 1	GAP ($C=16$)	—	0.13
Stage 2 – Blk 2	AP 16×16 , $S=8$, $C=16$	—	0.43
Stage 3 – Blk 3	AP 8×8 , $S=4$, $C=16$	—	0.47
Stage 3 – Blk 4	AP 4×4 , $S=2$, $C=16$	—	0.50
Stages 2–3	Bilinear interp. $\times 3$ branches	—	3.54
Stages 2–3	Pointwise addition ($\times 4$ branches)	—	0.52
Stage 4	PW Conv ($C_{\text{in}}=128 \rightarrow C_{\text{out}}=64$)	67.1	134.22
Total		80.21	167.08
Total (G)		0.08 GMACs	0.167 GFLOPs

Several observations are noteworthy:

- **Stage 4 dominates:** The final 1×1 convolution contributes 77 M FLOPs (67.3% of total), owing to the $2C = 128$ input channel depth after feature map concatenation. This motivated the 8-way parallel output-channel implementation ($128 \times 4 = 512$ parallel MACs) in Stage 4.
- **Stage 1 bottleneck mitigation:** Expanding Stage 1 from 64 to 128 parallel MACs reduces its execution time by half, balancing it with Stage 4 at approximately $\sim 410 \mu\text{s}$ each.
- **Stages 2–3 are lightweight:** The four DwDConv branches collectively account for only 9.44 M FLOPs (9.5%), confirming the efficiency of depthwise separable computation for contextual feature extraction.
- **Division-free pooling:** Using power-of-two window sizes (16×16 , 8×8 , 4×4) enables division by right-shift, reducing the AP hardware to pure accumulators.

6.4 Hardware Performance Metrics

Throughput and Latency

Metric	Value	Derivation
Operating Frequency ($f_{\max}$)	160 MHz	Post-implementation timing
Frame Rate (FPS)	774	Measured
Per-frame Latency	1.29 ms	$1/774$
Clock Cycles per Frame	215 609	$1.29 \text{ ms} \times 160 \text{ MHz}$
Total FLOPs per Frame	167 M	Table 10
Total MACs per Frame	80.21 M	Table 10
Sustained GFLOP/s	129.3	774×0.167
Sustained GMAC/s	61.92	774×0.04666

Table 11: Performance Metrics of the Proposed CTP Accelerator

Stage Execution Time Analysis

Stage	Parallel MACs	Cycles	Time (μs)
Stage 1 – PW Conv (1/4)	128	67 598	405.6
Stage 2 – DWD Blk 1+2	72	24 128	144.7
Stage 3 – DWD Blk 3+4	72	10,448	62.7
Stage 4 – PW Conv (1/2)	512	133 120	798.7
Pipeline Overhead and Pixel restoring	–	$\sim 5\,000$	~ 3
Total (Without Parallelization)	–	240 294	1.45 ms
Total (With Parallelization)	–	215 608	1.29 ms

Table 12: Execution Time Distribution Across Pipeline Stages

DSP Utilisation

Component	8-bit Multipliers Used	Purpose
Stage 1 PW Convolution (1/4)	128	Pixel $\times$ kernel multiplication
Stage 2 & Stage 3 (DW Conv + AP)	72	Depthwise dilated convolution, BN and ReLU
Stage 4 PW Convolution (1/2)	512	Pixel $\times$ kernel multiplication
Total DSP Usage	584 Multipliers (355 DSPs)	Out of 2800 DSP48E1 slices available on Virtex-7 VC707 (12.6% utilization)

Table 13: Multiplier and DSP Utilization Across Processing Stages

7 Comparative Analysis

This work presents the FPGA hardware acceleration of the Contextual Pyramid (CTP) module, originally proposed as part of CTPNet [13]—a lightweight real-time semantic segmentation architecture accepted for IEEE Transactions publication [13]. The hardware accelerator isolates and maps the CTP module’s multi-scale context aggregation onto a fully pipelined FPGA datapath, enabling independent benchmarking of the module’s inference efficiency on edge hardware.

Table 14 compares the proposed accelerator against representative full segmentation networks and contextual/attention modules deployed on FPGA or custom hardware.

Method	Params (M)	GFLOPs	Power (W)	FPS	Lat. (ms)	mIoU (%)
<i>Full networks (FPGA)</i>						
ENet [16]	0.36	3.8	7.3	31	32.3	58.3
SegNet [17]	29.5	286.0	25.0	7	142.9	59.1
ICNet [18]	26.5	28.3	10.5	68	14.7	67.7
ERFNet [19]	2.1	25.6	6.2	83	12.0	68.0
BiSeNetV2 [20, 21]	49.0	55.3	14.8	102	9.8	72.6
<i>Contextual modules</i>						
ASPP [14] [†]	2.1	12.4	—	—	~4.0	+3.8
PPM [15] [†]	5.5	35.2	—	—	~6.5	+4.1
LightASPP [23, 24]	0.72	1.8	3.1	—	5.2	+2.1
CBAM [25, 26]	0.50	0.8	1.4	—	2.8	+1.5
SE-Block [26, 27]	0.12	0.3	0.6	—	1.2	+0.9
<i>Proposed</i>						
CTP [13]	0.03	0.167	0.77	774	1.29	+3.0

Table 14: Comparison with full segmentation networks and contextual modules. mIoU: absolute for full networks; Δ mIoU for module-only entries.

7.1 Discussion

Parameter and computational efficiency. The CTP module, at 0.03 M parameters and 0.167 GFLOPs, is $10\text{--}70\times$ more parameter-efficient and $24\text{--}75\times$ leaner in computation than full networks such as ERFNet [19] and ICNet [18], and reduces FLOPs by $75\text{--}200\times$ relative to the heavyweight contextual modules ASPP [14] and PPM [15]. This reduction is achieved through Stage 1 channel compression and the use of depthwise dilated convolutions in place of standard 3×3 operations.

Throughput and latency. Operating at 774 FPS, the proposed accelerator outperforms every full-network FPGA result in Table 14 by a substantial margin— $6\times$ faster than BiSeNetV2 [20, 21] (102 FPS) and over $11\times$ faster than ICNet [18] (68 FPS). The 1.29 ms module-level latency provides considerable headroom for integration with a full encoder–decoder backbone while maintaining the real-time constraint of ≤ 33 ms at 30 FPS.

Power and on-chip resource utilisation. The 0.77 W on-chip power consumption is $4\text{--}19\times$ lower than full-network FPGA implementations, including ENet [16] (7.3 W), BiSeNetV2 [20, 21] (14.8 W), and SegNet [17] (25.0 W). With 15.3% LUT utilisation, over 84% of the Virtex-7 fabric remains available for the backbone and post-processing pipeline. The higher LUT count relative to SE-Block [26, 27] (11k) and CBAM [25, 26] (24k) is attributable to the substantially richer representational capacity of the CTP module, as reflected in its superior accuracy contribution.

Accuracy contribution. The CTP module delivers a +3.0% mIoU improvement, closely matching the gains of ASPP [14] (+3.8%) and PPM [15] (+4.1%) while requiring $75\text{--}200\times$ less computation. Against hardware-efficient alternatives, LightASPP [23, 24] (+2.1%) and CBAM [25, 26] (+1.5%), the CTP module provides a superior accuracy boost within a competitive FPGA footprint.

Overall assessment. The proposed accelerator establishes a Pareto-optimal operating point across accuracy, throughput, power, and resource utilisation. By delivering accuracy gains comparable to heavyweight contextual modules at a fraction of their computational cost—and at frame rates far exceeding all comparable FPGA implementations—the CTP module constitutes an effective plug-in context aggregation block for real-time semantic segmentation on resource-constrained edge platforms.

7.2 Summary

Table 15 consolidates the key performance and efficiency figures of the proposed CTP module FPGA accelerator.

Table 15: Summary of CTP module FPGA accelerator performance.

Metric	Value
Input dimensions	$128 \times 64 \times 64$
Trainable parameters	30 000 (≈ 0.03 M)
Total MACs per frame	80 M (0.08 GMACs)
Total FLOPs per frame	167 M (0.167 GFLOPs)
Operating frequency ($f_{\max}$)	160 MHz
Frame rate	774 FPS
Latency per frame	1.29 ms
Sustained throughput	129.3 GFLOP/s / 62 GMAC/s
Clock cycles per frame	215 605
DSPs used / available	355 / 2800 (12.6%)
LUTs used (est.)	$\sim 74\,558$ / 485 760 ($\sim 15.3\%$)
BRAMs used	1.65 MB / 4.5 MB (36%)
Pixel throughput	6.50 Mpixels/s
Efficiency (η_{DSP})	0.364 GFLOP/s/DSP

7.3 Future Aspects

Future work will focus on the full system-level integration of the CTP module into the complete CTPNet pipeline [13], targeting end-to-end deployment on the Virtex-7 VC707 platform. The integrated accelerator will be evaluated on the Cityscapes and CamVid benchmarks, which represent challenging real-world urban driving scenarios with dense multi-class annotations, to quantify the end-to-end mIoU contribution of the hardware-accelerated CTP module within the full encoder-decoder stack. In addition, design-space exploration will be conducted to investigate the impact of fixed-point quantisation and precision-reduced arithmetic on resource utilisation and segmentation accuracy, with the aim of further reducing on-chip power consumption and LUT occupancy without significant degradation in mIoU. Scalability of the proposed pipelined architecture to higher-resolution input tensors and to more recent UltraScale+ device families will also be examined, with a view toward establishing a generalised FPGA acceleration framework for lightweight contextual modules in real-time semantic segmentation systems.

References

- [1] N. Atif, S. Mazhar, M. Ameen, S. R. Ahamed, and M. K. Bhuyan, “SLICENet: An FPGA-based Efficient Semantic Segmentation Network for Edge Deployment,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 72, no. 9, pp. 1338–1342, 2025.
- [2] A. G. Howard, M. Zhu, B. Chen, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proc. IEEE CVPR*, pp. 4510–4520, 2018.
- [4] A. Howard, M. Sandler, G. Chu, et al., “Searching for MobileNetV3,” *Proc. IEEE ICCV*, pp. 1314–1324, 2019.
- [5] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, “BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation,” *Proc. ECCV*, pp. 325–341, 2018.
- [6] R. P. K. Poudel, S. Liwicki, and R. Cipolla, “Fast-SCNN: Fast Semantic Segmentation Network,” *Proc. BMVC*, 2019.
- [7] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” *Proc. ECCV*, pp. 801–818, 2018.
- [8] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, “Pyramid Scene Parsing Network,” *Proc. IEEE CVPR*, pp. 2881–2890, 2017.
- [9] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks,” *Proc. FPGA*, pp. 161–170, 2015.
- [10] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [11] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, “Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.

- [12] N. Atif, S. Mazhar, M. Ameen, S. R. Ahamed, and M. K. Bhuyan, “CTPNet: Achieving Real-Time Semantic Segmentation on Resource-Constrained Edge Devices for Autonomous Driving,” accepted for *IEEE transaction publication*, 2026.
- [13] N. Atif, S. Mazhar, M. Ameen, S. R. Ahamed, and M. K. Bhuyan, “CTPNet: Achieving Real-Time Semantic Segmentation on Resource-Constrained Edge Devices for Autonomous Driving,” *accepted for IEEE Transactions publication*, 2026.
- [14] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” in *Proc. ECCV*, 2018.
- [15] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, “Pyramid Scene Parsing Network,” in *Proc. IEEE CVPR*, 2017, pp. 2881–2890.
- [16] A. Paszke, A. Chaurasia, S. Kim, and E. Culurciello, “ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation,” *arXiv:1606.02147*, 2016.
- [17] V. Badrinarayanan, A. Kendall, and R. Cipolla, “SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, 2017.
- [18] H. Zhao, X. Qi, X. Shen, J. Shi, and J. Jia, “ICNet for Real-Time Semantic Segmentation on High-Resolution Images,” in *Proc. ECCV*, 2018.
- [19] E. Romera, J. M. Álvarez, L. M. Bergasa, and R. Arroyo, “ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation,” *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 1, pp. 263–272, 2018.
- [20] C. Yu *et al.*, “BiSeNet V2: Bilateral Network with Guided Aggregation for Real-Time Semantic Segmentation,” *Int. J. Comput. Vision*, vol. 129, pp. 3051–3068, 2021.
- [21] X. Zhang, J. Ye, and H. Fan, “FPGA-Based Accelerator for Real-Time Semantic Segmentation,” in *Proc. IEEE FPL*, 2022.
- [22] Y. Wang *et al.*, “LEDNet: A Lightweight Encoder-Decoder Network for Real-Time Semantic Segmentation,” in *Proc. IEEE ICIP*, 2019.
- [23] S. Mehta *et al.*, “ESPNet: Efficient Spatial Pyramid of Dilated Convolutions for Semantic Segmentation,” in *Proc. ECCV*, 2018.

- [24] Z. Yang, L. Xu, B. Li, R. Shi, and Q. Xu, “FPGA-Based Contextual Attention Accelerator for Semantic Segmentation,” in *Proc. IEEE FPL*, 2021.
- [25] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” in *Proc. ECCV*, 2018.
- [26] H. Liu, M. Gong, Z. Chen, and H. Li, “FPGA Acceleration of Attention Mechanisms for Semantic Segmentation,” in *Proc. IEEE ICCAD*, 2021.
- [27] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proc. IEEE CVPR*, 2018, pp. 7132–7141.